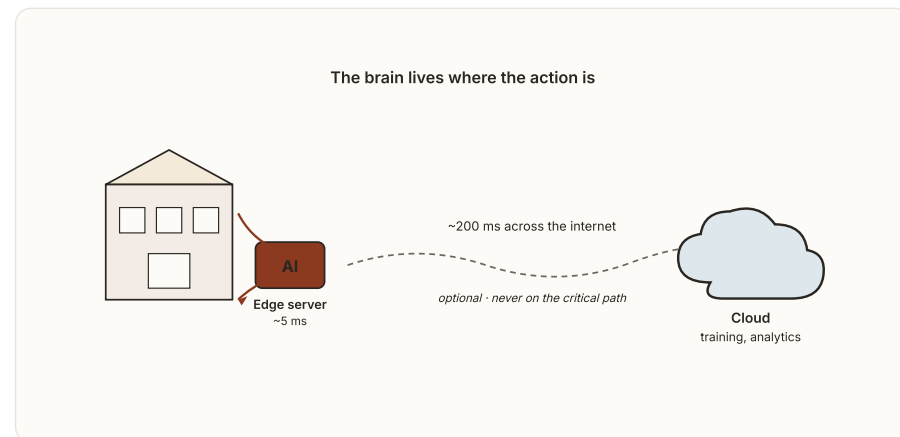


Lecture 1 — Introduction and Model-Based Systems Engineering

Standalone notes for the first lecture of D7065E. Read on its own or alongside

`lectures/lecture-1-introduction.md`.

Part 1 — Embedded Intelligence at the Edge



Local intelligence sits on the same network as the sensors and actuators it controls. The cloud helps with slow tasks, but the critical loop never depends on it.

A modern building, a modern car, a modern factory — none of these are simple machines anymore. Each contains hundreds of sensors and actuators, and a continuous stream of small decisions is needed to keep it running safely, efficiently, and comfortably. The branch of computer science that deals with making those decisions on the same network as the physical system, rather than in a remote data centre far away, is called **embedded intelligence at the edge**.

The phrase has two halves worth unpacking.

Embedded means the software lives inside or right next to the physical system it controls. The control logic of a car's anti-lock brakes lives in the car. The thermostat's brain lives in the

thermostat itself. There is no question of phoning home to ask permission. The software is part of the thing.

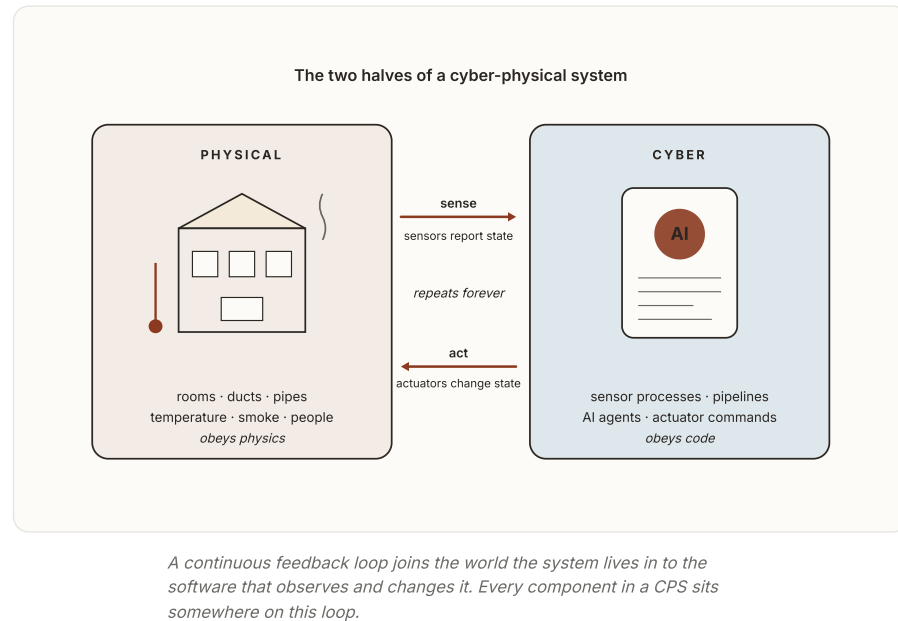
At the edge means on the local network — close to the sensors and the actuators — not in the cloud. A useful image: the difference between a self-driving car that can stop itself when something runs into the road, and one that has to ask a server in another country whether to brake. The first one is safe. The second one is a horror film waiting to happen.

Two ideas follow naturally. First, decisions happen fast and locally, in milliseconds, because the physical world cannot wait for a round trip across the internet. Second, the system keeps working when the wider network does not. If the building's broadband line is cut during a storm, the fire detector still detects fires.

The course's running project, autonomous building control, makes both points concrete. A simulated building called BuildSim provides rooms, sensors, and actuators. The job is to instrument that building with software that observes its state, decides what to do, and changes the state — all running on the same network as the building itself, without depending on any cloud service to make the decisions. Cloud services can help with slow tasks, like overnight model training, but they are never on the critical path.

Before any code is written, a discipline called Model-Based Systems Engineering is applied: the system is fully specified using structured diagrams and tables. The specification is reviewed and approved at week three. The approved design becomes the basis for both implementation and the final oral examination.

Part 2 — A Building as a Cyber-Physical System



The two halves of a CPS

A **cyber-physical system**, abbreviated CPS, is a system where computation and physical processes are tightly intertwined. Two halves live side by side and communicate forever.

The **physical half** is everything you can touch. In a building, that means rooms, walls, doors, HVAC ducts, ventilation fans, smoke and air, temperature gradients, water pipes, electrical loads, and the people walking around inside. Physical state evolves according to the laws of physics and the patterns of human behaviour. The temperature in a room does not stop changing just because no computer is watching.

The **cyber half** is the software. Sensor processes that read the physical state and report it. Data pipelines that store the readings. AI agents that reason about what should happen next. Actuator processes that issue commands. The cyber half is fast and flexible but has no direct access to physics — it only knows what sensors report, and it only acts through what actuators can do.

A useful image: the physical half is the room, the cyber half is the brain. The brain cannot feel the temperature directly. It can only ask the thermometer (a sensor) what the temperature is, and it can only warm the room by telling the heater (an actuator) to turn on. Everything in between is just words and electricity.

The continuous feedback loop

A CPS is defined by the loop that joins the two halves. The cycle runs forever, and every component sits somewhere on it.

1. Physical world produces a state (smoke level rises)
2. Sensor measures the state (smoke = 0.82)
3. Software reasons about the measurement (this looks like a fire)
4. Actuator changes the world (sprinkler on)
5. World responds to the change (smoke level drops)
6. Sensor measures the new state (smoke = 0.4)
7. Software notices and adapts (working; consider easing off)
8. (Back to step 1.)

Riding a bicycle is the simplest everyday version of this loop. You sense your balance (eyes and inner ear are the sensors). You decide whether you're leaning too far left or right (the brain is the reasoner). You correct (your muscles are the actuators). And your balance changes — the world responds. The loop runs many times per second, automatically, and if any part of it breaks, you fall over.

A CPS is a bicycle for an entire building. The loop has to be fast enough that the physics can't run away (a fire-suppression loop in well under a second, a climate-control loop comfortable with one update every few minutes). It has to be reliable enough to keep working when a sensor

crashes, a network drops, or a model gives the wrong answer. And it has to be correct enough that a wrong decision does not start a fire, lock people in during an emergency, or freeze a server room.

Automation versus autonomy

A grandfather clock is automated. Every hour, it strikes the hour. It follows a fixed rule that was wound up once and changes nothing. Automation is predictable and easy to verify but brittle — the rule has to anticipate every situation, and any situation it didn't anticipate becomes a bug.

A modern smart thermostat is autonomous. It knows that the building is empty on Sunday morning, that the weather forecast is mild, that the family is on holiday. It predicts that nobody will want a warm room for the next eight hours. It learns from history. It adapts as patterns change.

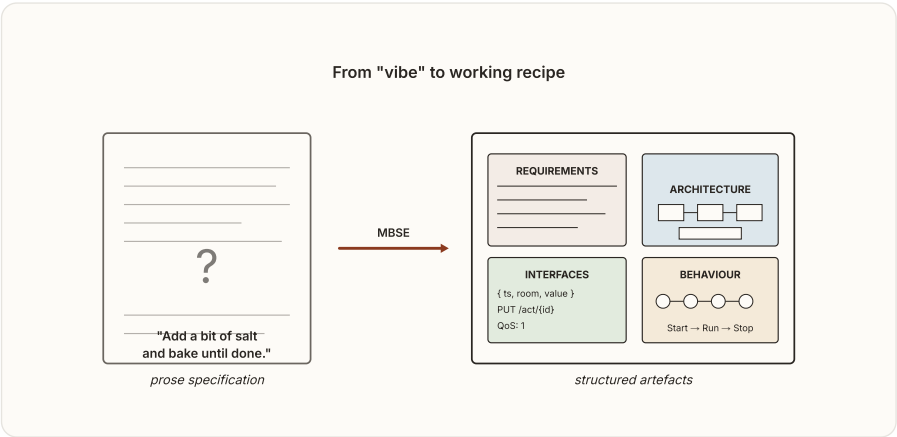
The difference is bigger than it sounds. Two ways to think about it:

AUTOMATION	AUTONOMY
if temp < 20°C: turn on heater	"It's 9 a.m. on a Tuesday, the office will fill in 30 minutes, outdoor temp is dropping, pre-heat now so the room reaches 22°C by then."
One rule, fixed forever	Many models, adapting over time
Predictable but brittle	Smarter but less predictable

Real-world autonomous building systems include Johnson Controls' OpenBlue, Siemens Designo CC, and the cooling controller that DeepMind built for Google's data centres — which reduced cooling-energy consumption by about 40 percent by reasoning over far more variables than a rule-based controller can hold.

The course's autonomous building control project asks for the autonomous version, not the automated one. Threshold rules ("if smoke > 0.7, alarm") are not sufficient. A real ML model or LLM reasoning step is required.

Part 3 — Specification Before Code: Model-Based Systems Engineering



MBSE replaces ambiguous documents with structured, precise models. Two engineers reading the same model build the same mental picture.

Why prose breaks down

The default way to design software is to write a Word document describing what should be built, hand it to the developers, and hope they get the same picture from it.

This almost always fails. Prose is ambiguous, and the ambiguity is invisible until somebody tries to act on the document. Two engineers reading the same paragraph form different mental pictures. The product manager imagined a database; the developer built a spreadsheet. The architect assumed the broker held messages for a minute; the operations engineer configured it to drop them after a second. Three weeks into implementation, somebody discovers a contradiction. Six weeks in, the design and the code have parted ways completely.

A useful image: writing prose specifications is like cooking from a recipe that says "add a bit of salt and bake until done." Five cooks following this recipe produce five different dishes. The recipe didn't capture the actual instructions; it captured a vibe.

A working recipe says: "1.5 teaspoons of salt, bake at 180°C for 35 minutes." Five cooks following this recipe produce the same dish. The difference between a vibe and a working recipe is

precision.

What MBSE is

Model-Based Systems Engineering, abbreviated MBSE, replaces prose documents with structured, precise models of the system. A model is unambiguous — either it specifies a thing or it doesn't. Two engineers reading the same model build the same mental picture.

The models are not single documents but a small collection of structured artefacts:

- A **requirements table** listing every requirement with a unique ID, a type, a priority, and an acceptance criterion.
- An **architecture diagram** with labelled boxes for every component and labelled arrows for every connection.
- **Interface specifications** — JSON schemas for every message, REST endpoints with request and response shapes, MQTT topic names and payload formats.
- **Behaviour models** — sequence diagrams showing message exchanges over time, state machine diagrams showing what each component does in each state.
- A **validation matrix** linking each requirement to the test that proves it works.

A useful image: an architect designing a house doesn't write a 200-page essay about the house. They draw blueprints. The blueprints have measurements, labels, and conventions that any contractor in the world can read the same way. MBSE applies the same idea to software systems.

Why this matters more for CPS

Two engineers building a typical web application can sometimes get away with ambiguity, because the consequences of a misunderstanding are usually a wrong-looking page that gets fixed in the next sprint.

A cyber-physical system is different. The interactions between physical processes and software components are complex, timing-sensitive, and have safety consequences. A sequence diagram showing a smoke sensor publishing a reading, a data pipeline storing it, an anomaly model evaluating it, and a safety agent commanding a sprinkler makes the design concrete in a way that no paragraph can. A state diagram of the sensor process specifies exactly what happens when the network drops and what happens when it recovers. These artefacts catch bugs that prose hides.

The MBSE process

The work flows through six activities. They are not strictly sequential — work loops back as understanding deepens — but they are ordered in importance and dependence.

Requirements analysis. Write down what the system must do as testable statements, each with a unique ID. Three kinds appear: **functional** ("detect fire conditions within 30 seconds"), **non-functional** ("survive a sensor-process crash without losing data"), and **regulatory** ("comply with Swedish BBR fire protection requirements"). A useful image: requirements are the customer's wishlist, formalised so it cannot be misread. "Make the cake delicious" is not a requirement. "The cake must be 30 cm in diameter, contain no peanuts, and be ready by 6 p.m." is.

Functional decomposition. Take each high-level requirement and break it into the smaller operations needed to satisfy it. "Detect fire" decomposes into collecting smoke and temperature readings, validating them, applying a detection model, raising an alert, commanding sprinklers, and notifying occupants. A useful image: decomposing a recipe. "Make a cake" decomposes into "measure flour, beat eggs, mix, pour into tin, bake at 180°C for 35 minutes, cool, frost." Each step is small enough to assign to a worker.

Architecture design. Decide what software components exist, how they're organised, and where they run. A component diagram shows the parts. A deployment view shows which machine each part lives on. A useful image: the floor plan of a house. Walls, rooms, doors — but no furniture yet.

Interface design. Specify exactly how components talk to each other. The REST endpoint URL and the request and response shapes. The MQTT topic name and the payload format. The protocol, the message size, the rate. Ambiguity here is the most expensive kind. Two components that look connected on the diagram but disagree about the message format will work in isolation and fail when they meet. A useful image: a power plug and a wall socket. If the prongs don't match the holes, the lamp doesn't light. Better to discover that mismatch on paper than on the day the lamp is supposed to ship.

Behaviour modeling. Capture how the system acts over time. A **sequence diagram** is a comic strip with software components as the characters — it shows one specific scenario, with messages going back and forth in order. A **state machine** is a map of the moods a component can be in — `Starting` , `Running` , `Reconnecting` , `Stopped` — and the events that flip it between them. Both are precise; both expose timing bugs that static diagrams hide.

Validation. Close the loop by linking every requirement to a design element that satisfies it and a test case that verifies it. If a requirement has no owning component, the design has a gap. If it has

no test, the requirement cannot be enforced. A useful image: tasting the soup before serving. Recipes that produce dishes nobody tests are recipes that produce surprises.

Why design happens before code

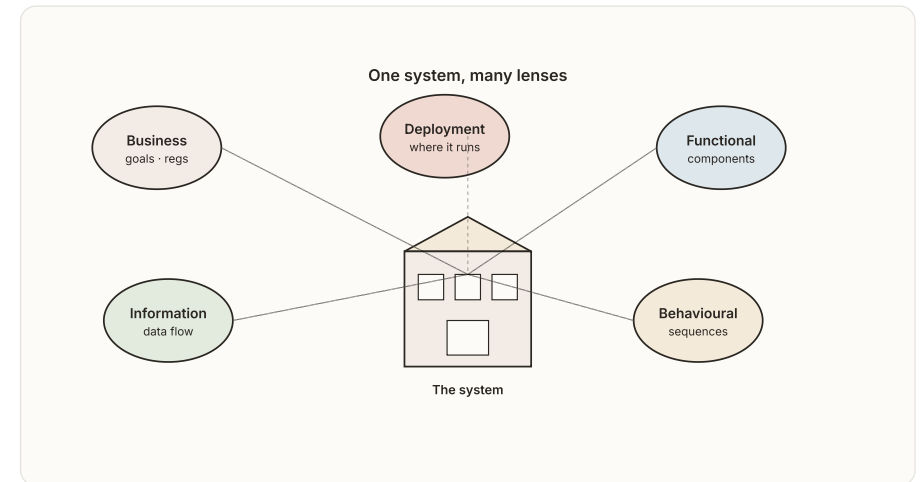
Errors discovered during design cost minutes. Errors discovered during implementation cost days. Errors discovered after deployment cost weeks, or — in safety-critical systems — lives.

The cost curve is steep, and the MBSE process catches as many errors as possible while they are still cheap.

A second reason has emerged more recently. When code is generated with AI tools, the quality of the output is bounded by the quality of the input. A vague prompt produces clean-looking code that satisfies the prompt but not the underlying problem. A precise specification produces working code that solves the actual problem. The specification is also the test contract — the same artefact that describes the system also describes how to know whether it was built correctly.

A useful image: imagine asking a contractor to "build me a house." You will get a house, technically. It will not be the house you wanted. Now imagine handing the contractor a complete set of blueprints with measurements, materials, and electrical layout. You will get the house you wanted. The contractor is talented in both cases. The difference is the contract.

Part 4 — Architecture Viewpoints



Different stakeholders need different views — no single diagram serves them all. Each viewpoint filters out everything not relevant to one specific concern.

Why one diagram is never enough

A single diagram cannot serve every audience. A developer writing code needs to see software components. An operator deploying the system needs to see machines and networks. A safety engineer needs to see failure modes. A building manager needs to see which components are responsible for regulatory compliance.

If one diagram tries to show everything at once, it becomes unreadable — a bowl of spaghetti with too many arrows. If it focuses on one perspective, it hides everything else. The solution is multiple diagrams, each tailored to one specific concern.

A useful image: a house seen by different people. The future home-owner needs a floor plan to imagine living there. The electrician needs a wiring diagram. The plumber needs a pipe diagram. The structural engineer needs a load-bearing-walls diagram. All four describe the same house, but each one filters out everything that doesn't matter for that audience. Trying to draw all four on one sheet of paper produces a mess.

The same idea applied to software systems is called **architecture viewpoints**. The concept is formalised in the international standard IEEE 42010, in Kruchten's influential 4+1 View Model (1995), and in enterprise frameworks like ArchiMate.

What a viewpoint actually is

A **viewpoint** is a way of looking at the system that filters out everything not relevant to one stakeholder concern. Each viewpoint is a lens. Each catches design errors the others miss.

A useful image: a subway map and a street map of the same city. The subway map shows lines and stops — everything else is suppressed. The street map shows streets and addresses — the subway is barely visible. Both are correct. Neither one is enough on its own. A tourist needs both.

ArchiMate's three layers

The ArchiMate framework organises architecture across three layers. Each layer answers a different stakeholder question.

LAYER	CONCERN	BUILDING CONTROL EXAMPLES
Business	Processes, actors, goals, regulations	The building manager monitors safety; the building must comply with BBR fire code
Application	Software components, data flows, interfaces	An AI agent, an anomaly detector, a data pipeline, the BuildSim API client
Technology	Infrastructure, devices, containers, networks	Docker containers on an edge server, a GPU server, an MQTT broker, a TimescaleDB instance

The same system can be described at all three layers, and each description is useful to a different audience. A senior manager cares about the business layer. A developer cares about the application layer. A site reliability engineer cares about the technology layer.

The five required viewpoints for a CPS architecture document

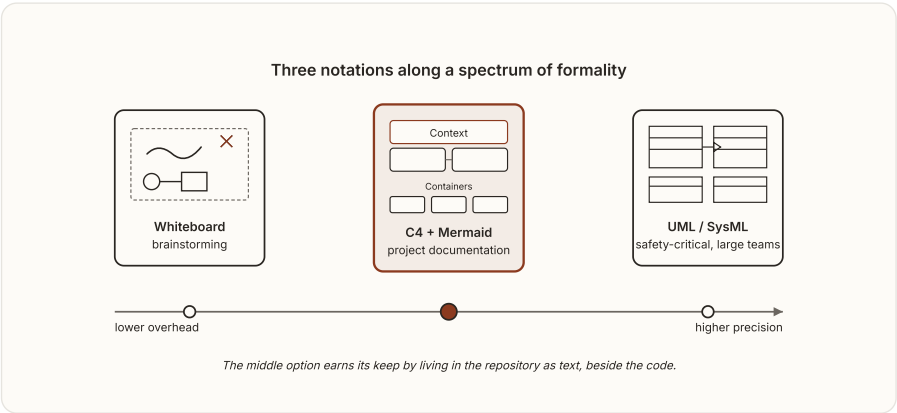
A complete architecture description for a building control system contains five viewpoints. Each answers a different question.

VIEWPOINT	QUESTION IT ANSWERS	WHAT IT SHOWS
Context	What interacts with the system?	The system as a single box, with users and external systems around it
Functional	What are the parts, and how do they connect?	All components inside, with their interfaces
Information	What data exists, and how does it flow?	Data models, storage, transformations
Behavioral	What happens when a specific event occurs?	The sequence of messages for one scenario
Deployment	What runs where?	Containers, hardware, network topology

Each viewpoint is the lens that catches a different kind of mistake. A correct functional view can hide a broken deployment view — two components that look connected on paper might actually require a network link that does not exist. A clean data flow can mask a behavioural problem — a sequence diagram of a specific scenario can reveal a timing issue that no static diagram exposes. A complete business view can expose a compliance requirement that no component has been assigned to satisfy.

A useful image: five different home inspections of the same house. The structural inspector checks the foundation. The electrical inspector checks the wiring. The plumbing inspector checks the pipes. The roof inspector checks for leaks. The pest inspector checks for termites. Each one is looking for something different. Skipping any of them means certain defects will only be discovered after you move in.

Part 5 — Modeling Notations



Three notations along a spectrum from informal to industrial. C4 with Mermaid is the chosen middle ground — enough precision to be useful, light enough to stay in version control.

Picking the right notation is partly about precision and partly about overhead. Different tools sit at different points on that trade-off.

Whiteboard sketches

A pen on a whiteboard is the right tool for early brainstorming. No formalism, no syntax to check, no learning curve. The trade-off is zero precision and zero versioning. Sketches are excellent for thinking; they are inadequate as the final architecture document.

A useful image: a napkin sketch over coffee captures an idea. It does not survive being passed across the office.

UML and SysML

The **Unified Modeling Language**, abbreviated UML, has been the industry standard for modelling software systems since the late 1990s. It defines fourteen diagram types: class diagrams, sequence diagrams, state machine diagrams, activity diagrams, deployment diagrams, and many more.

SysML extends UML with diagrams for systems engineering — requirements traceability, physical constraints, hardware-software integration. UML and SysML are used in aerospace, defence, and automotive industries with heavyweight tools like Cameo Systems Modeler and Eclipse Papyrus.

Both have a steep learning curve and heavy tooling. For a small team working over a few weeks, the formalism adds more overhead than value. The strengths of UML and SysML — strict typing, simulation, automated code generation — pay off for a 200-engineer Airbus programme, not for a course project.

A useful image: UML is an industrial CNC machine. SysML is a CNC machine with extra accessories. Both are remarkable. Neither is what you want for a weekend furniture project, where a screwdriver and a level get the job done.

The C4 model

The **C4 model** was proposed by Simon Brown specifically because UML felt too complex for most teams. C4 has exactly four levels of abstraction.

LEVEL	WHAT IT SHOWS
1. Context	The system as a single box surrounded by users and external systems
2. Containers	The deployable units inside (processes, Docker containers, databases)
3. Components	The internal structure of one container
4. Code	Class-level detail (rarely drawn manually)

C4 is technology-agnostic and focuses on structure and relationships. Levels 1 and 2 are sufficient for most software projects. Level 3 is useful when one container is complex enough to warrant its own diagram, such as an AI agent with multiple internal subsystems.

A useful image: C4 is Google Maps. Zoom out to see the country (Context). Zoom in to see the city (Containers). Zoom in further to see the streets (Components). Zoom all the way in to see individual buildings (Code). Same map, four levels of detail, each appropriate to a different question.

Diagramming tools

The recommended diagramming tool for this course is **Mermaid**. Diagrams are written as plain text inside Markdown files, versioned with git alongside the code, and render automatically when the Markdown is viewed on GitHub.

The biggest advantage of Mermaid is that the diagrams live in the same repository as the code, get reviewed in the same pull requests, and stay synchronised with the system. A diagram pasted as a PNG into a Word document drifts out of sync within a week.

A useful image: a recipe taped to the inside of the kitchen cupboard, edited as the cook refines the recipe. The dish stays the recipe. The recipe stays the dish.

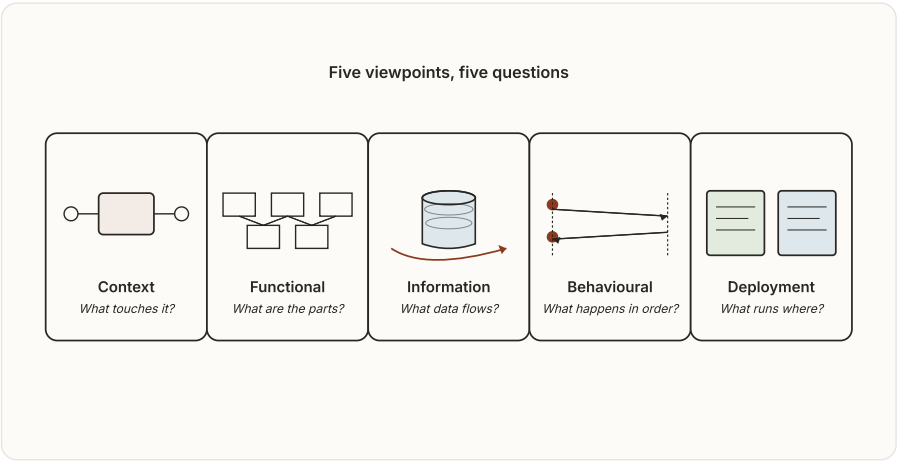
Other tools have specific niches. **draw.io** is open and free for more complex visual layouts. **Excalidraw** produces hand-drawn-looking diagrams useful for sketches and brainstorming. Neither matches Mermaid's advantage of living inside the repository as text.

Comparing the choices

ASPECT	WHITEBOARD	C4 + MERMAID	UML / SYSML
Learning curve	None	About an hour	Days to weeks
Precision	Low	Medium	High
Tooling	Pen	Markdown editor	Cameo, Papyrus
Best for	Brainstorming	Project documentation	Safety-critical, large teams

For this course, C4 with Mermaid is the chosen middle ground.

Part 6 — The Five Viewpoints in Practice

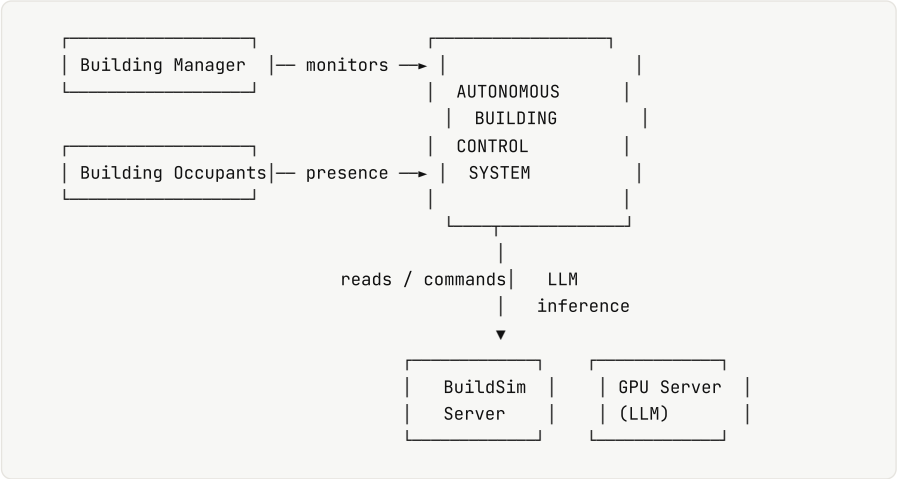


Each viewpoint asks a different question — and each catches a kind of design error the others would hide.

Each viewpoint introduced in Part 4 looks like something specific when drawn. The following examples apply each viewpoint to building control.

Context view (C4 Level 1)

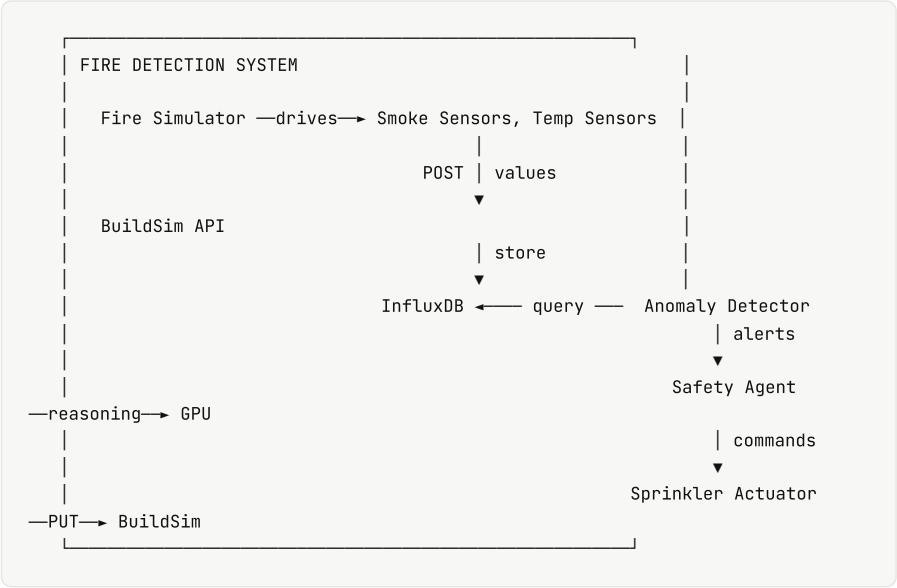
The context diagram shows the entire system as a single box, surrounded by every person and every external system it interacts with. Nothing internal is shown.



The diagram answers one question only — who or what touches the system from outside. A useful image: a country on a globe, with arrows showing where its trade goes. Nothing about cities is shown yet.

Functional view (C4 Level 2)

The container diagram opens up the single box from the context view. Inside are the deployable units — each box typically maps to one Docker container.



Two different use cases lead to two different container diagrams. A fire-detection system needs sensors, an anomaly detector, a safety agent, sprinkler actuators, and an LLM. An HVAC optimisation system needs temperature and occupancy sensors, an MQTT broker, a time-series database, a forecasting model, an HVAC controller, and HVAC actuators. The same physical building, the same BuildSim, but two different functional decompositions.

A useful image: zooming into the country on the globe and seeing its cities. Each city is one container. Lines between them are roads — the interfaces.

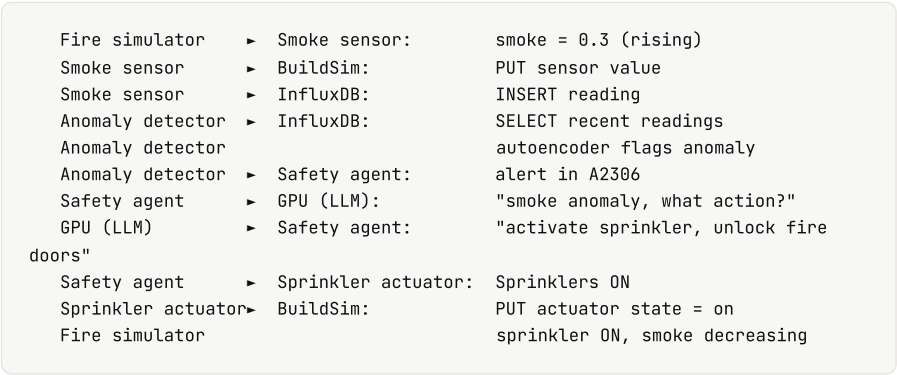
Component view (C4 Level 3)

When one container is complex enough, it gets its own diagram showing its internal structure. A safety agent might contain a BuildSim API client, a set of tool definitions, an agent memory, a reasoning chain implementing the ReAct pattern, and a safety guardrail module. Each is internal to the agent and invisible from the outside.

A useful image: zooming further into one city and seeing its streets. The streets are not visible at the city-level view, but they exist.

Behavioral view (sequence diagram)

A **sequence diagram** is a comic strip with software components as the characters. It shows one specific scenario as a series of messages, in order.

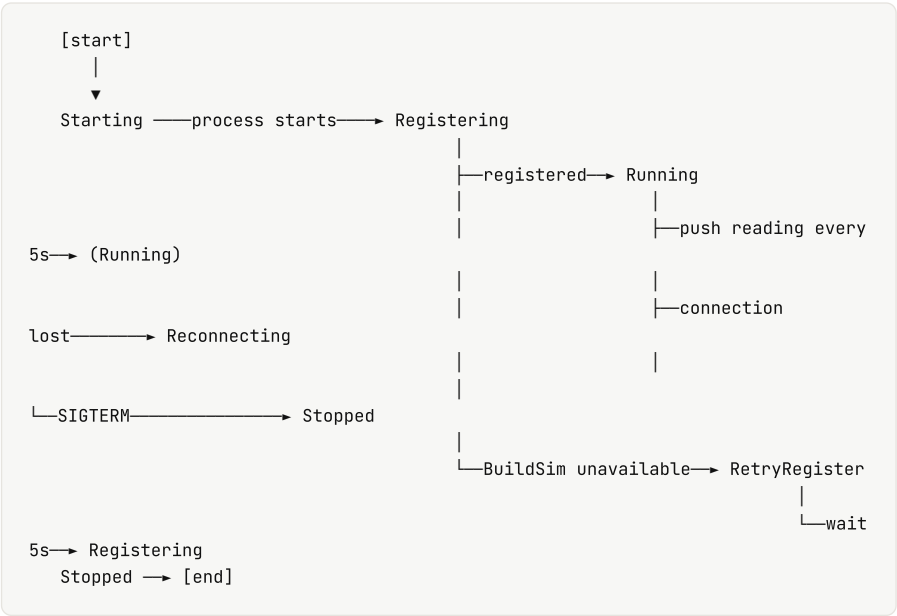


Each line is one frame of the comic. The order of the lines is the order of events. Sequence diagrams are the right tool when the question is "what happens, in what order, for this scenario?" They expose timing issues that no static component diagram can.

A useful image: a film strip. Each frame is one moment in time. Played in order, the strip tells a story.

State machine diagram

A **state machine** shows every state a component can be in and every event that moves it between states. The lifecycle of a sensor process might look like this:



A useful image: a traffic light. It has states (red, yellow, green) and events that flip it (timer expires, pedestrian button pressed). The state machine is the rulebook for the lamp.

Every transition in the diagram is a piece of code that handles a specific event. If the diagram is complete, the implementation can be written almost mechanically from it.

Information view (requirements table)

The information viewpoint is sometimes a table rather than a diagram. The requirements table lists every requirement with its ID, type, statement, priority, and acceptance criterion.

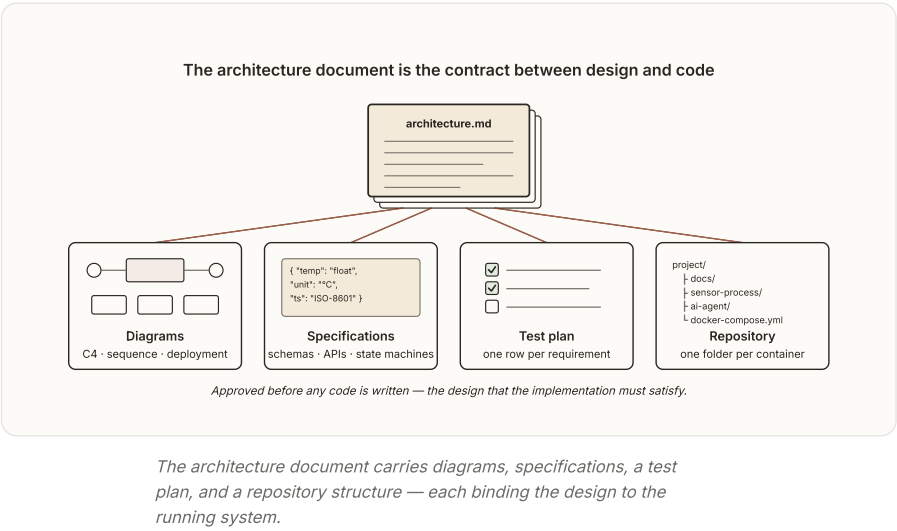
ID	TYPE	REQUIREMENT	PRIORITY	ACCEPTANCE CRITERION
FR-01	Functional	Detect fire within 30 seconds	Must	Anomaly detector flags within 30 s
FR-02	Functional	Activate sprinklers in affected rooms	Must	Actuator state changes to "on"
FR-03	Functional	Compute evacuation routes avoiding fire	Must	Route excludes fire rooms
NFR-01	Non-functional	Recover from sensor crash within 60 s	Must	New reading within 60 s of kill
NFR-02	Non-functional	False positive rate below 5 %	Should	Evaluated on 24 h of normal data
REG-01	Regulatory	Fire doors close per BBR timing	Must	Door responds within 5 s

Every requirement traces to a test case. FR-01 maps to a test called

`test_fire_detection_latency`.

A useful image: an invoice with line items. Each line is a deliverable. The total at the bottom is the contract.

Part 7 — The Architecture Document



The architecture document is the contract between design and implementation. It is reviewed and approved before any code is written. It contains the five viewpoints described above, plus the specifications below.

Diagrams and tables

The document includes:

- A **context diagram** (C4 Level 1) showing the system boundary, users, and external systems.
- A **container diagram** (C4 Level 2) showing every deployable unit and how they connect.
- A **requirements table** mapping each requirement to a priority, acceptance criterion, and test case.
- A **data flow diagram** showing how sensor data moves from measurement through storage to decision.
- One or more **sequence diagrams** for the key scenarios.
- A **deployment diagram** showing what runs where.

Specifications

Beyond the diagrams, the design specification covers four additional artefacts.

Data models. JSON schemas for every message that crosses a boundary between components. Each schema specifies field names, types, units, and constraints. A useful image: a customs declaration form. Every field on the form has a defined meaning, and every package crossing the border must complete it correctly.

API contracts. Every REST endpoint and every MQTT topic in the system, with request and response shapes, status codes, QoS levels, and retention semantics. The plug-and-socket of the design — the prongs that have to match between components.

State machines. For the AI agent and for every component whose lifecycle matters, a state machine specifying every state and every transition. Especially important for error handling — the state machine is where you specify what happens when the network drops, when a sensor crashes, when a command fails.

ML model specifications. For each model, the input feature vector, the output, the source of training data, the training procedure, and the evaluation metrics that determine whether the model is good enough to deploy.

Test plan

The test plan is written **before** implementation, not after. It links each requirement to one or more test cases. Each test case describes the initial state of the system, the stimulus that drives the test, the expected response, and the pass-or-fail criteria.

A useful image: a wedding-planner's checklist. Every item on the wishlist has a corresponding "yes/no — was it delivered?" tick. The wedding cannot be declared successful until every box is ticked.

Repository structure

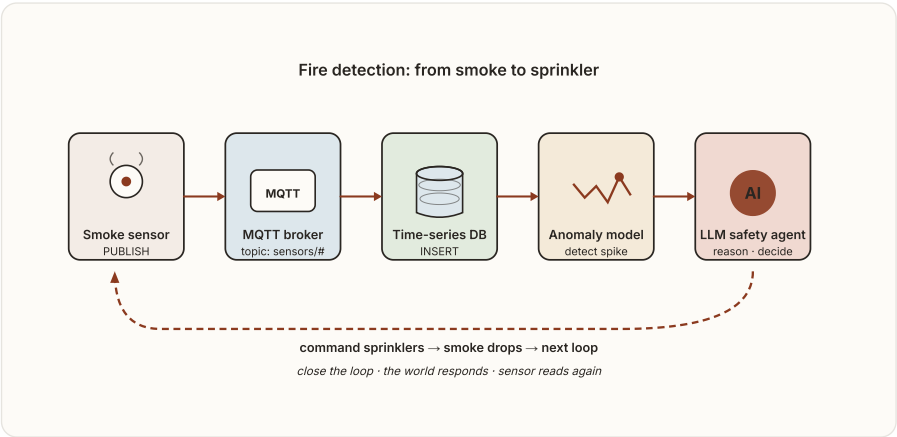
A consistent repository structure keeps the design and the code in sync. One folder per deployable container, plus a docs folder for the architecture document, plus a top-level docker-compose file that brings everything up.

```
project-root/
├─ docs/
│   └─ architecture.md
│   └─ requirements.md
│   └─ test-plan.md
├─ sensor-process/
│   └─ Dockerfile
│   └─ src/
├─ ai-agent/
│   └─ Dockerfile
│   └─ src/
├─ actuator-process/
│   └─ Dockerfile
│   └─ src/
├─ docker-compose.yml
└─ README.md
```

The structure mirrors the C4 container diagram. If the diagram shows five containers and the repository has one folder with everything jammed together, the design and the code have parted ways.

A useful image: the kitchen of a restaurant. Each station (grill, salad, dessert) has its own workspace, equipment, and chef. The menu (the design) is reflected in the kitchen layout (the code structure). A messy kitchen means slow, error-prone service.

Part 8 — A Worked Example: Fire Detection End-to-End



Every component of the fire-detection scenario, in the order they participate. The dashed arrow closes the loop — the sprinkler changes the world, the next sensor reading reflects it.

To make the abstract MBSE process concrete, follow a fire-detection system through every step.

Step 1 — Requirements

The system must detect fire within 30 seconds, activate sprinklers in the affected rooms, compute evacuation routes that avoid the fire, recover from a sensor crash within 60 seconds, keep false positives below 5 percent, and ensure fire doors close within the timing specified by the Swedish BBR fire code. Each becomes a row in the requirements table with a unique ID, a priority, an acceptance criterion, and a target test case.

Step 2 — Functional decomposition

"Detect fire" decomposes into eight operations: collect smoke, CO, and temperature readings continuously; validate the readings; compute features (rolling means, gradients); apply an anomaly model; escalate positive readings to a language-model agent for severity assessment;

decide actions (sprinkler, fire-door, evacuation route); apply commands through actuator processes; log the decision and the outcome.

Each operation maps to a software component.

Step 3 — Architecture (Container diagram)

The components: a fire simulator that drives the physical model, sensor processes that read from it, an MQTT broker that decouples sensors from consumers, a time-series database that stores history, an anomaly detector running the Isolation Forest model, a safety agent that calls the LLM and chooses actions, sprinkler and fire-door actuator processes, and a dashboard process that visualises building state. Each is one container; everything is described in a single `docker-compose.yml`.

Step 4 — Interfaces

Every connection is specified exactly. Sensors publish to MQTT topics named `sensors/{level}/{room}/{type}` with the payload `{ts, sensor_id, room, level, type, unit, value}` at QoS 1. A consumer subscribes to `sensors/#` and writes each reading to the `readings` hypertable in TimescaleDB. The anomaly detector queries with `SELECT value FROM readings WHERE room = $1 AND type = 'smoke' AND ts > now() - INTERVAL '60 seconds'`. The agent issues actuator commands via `PUT /api/actuators/{id}/state` with body `{"state": "on"}`.

Nothing is left to interpretation.

Step 5 — Behaviour

A sequence diagram walks through the fire scenario: smoke rises in A2306, the sensor publishes, the consumer inserts, the anomaly detector flags, the agent calls the LLM, the agent commands the sprinkler, the sprinkler updates both BuildSim and the simulator, and the next sensor cycle shows reduced smoke. A state machine for the sensor process specifies registration, normal operation, network-failure handling, and reconnection.

Step 6 — Validation

Every requirement maps to a test:

REQUIREMENT	TEST	PASS CRITERION
FR-01	test_fire_detection_latency	Less than 30 s from smoke spike to actuator command
FR-02	test_sprinkler_fires	Actuator state changes to "on" within 5 s of detection
NFR-01	test_sensor_crash_recovery	New reading within 60 s of killing the sensor
NFR-02	test_false_positives	Below 5 percent on 24 hours of normal data

Each test runs against the actual deployed system, not a mock.

The entire fire-detection system — requirements through validation — is captured in roughly twenty pages of structured artefacts: tables, diagrams, and JSON schemas. None of it depends on any reader interpreting prose.

Part 9 — Vocabulary Reference

Every term used in this chapter, defined.

TERM	DEFINITION
CPS (Cyber-Physical System)	A system in which computation and physical processes are tightly coupled through a continuous feedback loop
Embedded intelligence	Software that runs inside or beside the physical system it controls, not in a remote data centre
Edge	The portion of a network physically close to the data source; the opposite of cloud
Sensor	A component that measures a physical quantity and reports it as data
Actuator	A component that performs a physical action when commanded
Feedback loop	A control structure in which the output of a process influences the next input
Automation	Behaviour produced by a fixed rule that does not adapt to context
Autonomy	Behaviour that adapts to context using data, prediction, and learning
MBSE (Model-Based Systems Engineering)	A methodology that replaces prose documents with structured, precise models
Requirement	A testable statement of what the system must do
Functional requirement	A requirement about behaviour: what the system does
Non-functional requirement	A requirement about quality: how well the system does it
Regulatory requirement	A requirement imposed by an external standard or law
Functional decomposition	Breaking a high-level requirement into smaller operations that satisfy it

TERM	DEFINITION
Architecture	The high-level structure of components and their relationships
Interface	A precise specification of how two components communicate
Sequence diagram	A diagram showing the messages exchanged between components over time, for one scenario
State machine	A diagram showing the discrete states of a component and the events that cause transitions
Validation	Demonstrating that a requirement has been met by a design element and a test
Viewpoint	A perspective on the system that filters out everything not relevant to one stakeholder concern
IEEE 42010	The international standard defining architecture descriptions through multiple viewpoints
ArchiMate	An enterprise architecture framework with three layers: business, application, technology
4+1 View Model	An influential set of architectural views proposed by Kruchten in 1995
C4 Model	A simple architectural notation with four levels: Context, Containers, Components, Code
UML (Unified Modeling Language)	A standard modeling language with fourteen diagram types
SysML	An extension of UML for systems engineering, used in aerospace and defence
Mermaid	A text-based diagramming tool that renders inside Markdown

TERM	DEFINITION
Container (Docker)	A packaged, isolated runtime that bundles a service with all its dependencies
Docker Compose	A tool for running multiple containers together, configured by a YAML file
Architecture document	The contract between design and implementation, produced by the MBSE process

Part 10 — Summary in Five Sentences

1. A building is a cyber-physical system in which a continuous feedback loop joins the physical world and the software that observes and changes it; the loop must be fast enough that the physics cannot run away, reliable enough to survive component failures, and correct enough to avoid harm.
2. The right place for the software's brain is the edge, close to the physical system, because latency, reliability, privacy, and bandwidth all favour local processing over remote.
3. Useful systems are autonomous, not merely automated: they reason about context, learn from data, and adapt over time rather than following a single fixed rule.
4. The path from idea to working code runs through Model-Based Systems Engineering, in which structured artefacts — requirements tables, architecture diagrams, interface specifications, behaviour models, and validation matrices — replace prose documents that cannot be analysed or tested.
5. A complete architecture description requires multiple viewpoints (context, functional, information, behavioural, deployment), because no single diagram can serve every stakeholder, and each viewpoint catches errors the others hide.

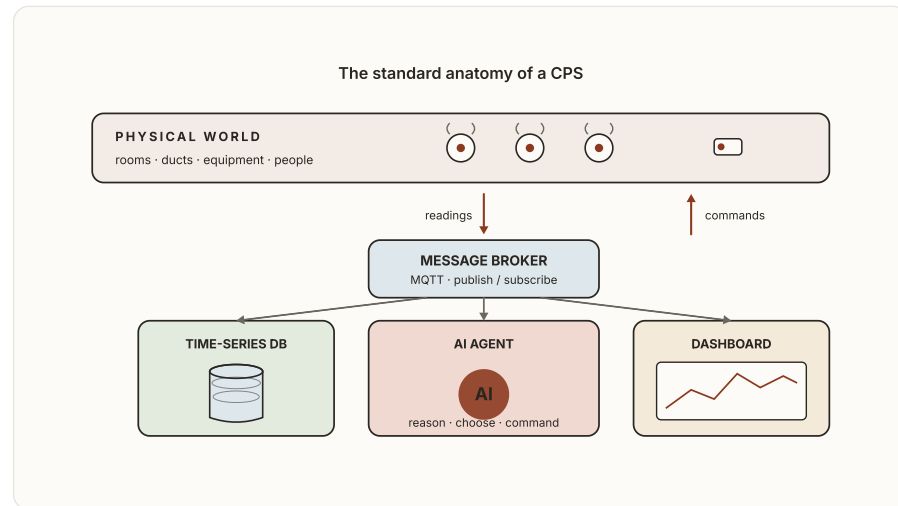
These five ideas are the foundation for everything that follows.

Lecture 2 — Edge Intelligence & CPS Architecture

Standalone notes for the second lecture of D7065E. Read this on its own or alongside

`lectures/lecture-2-cps-architectures.md`.

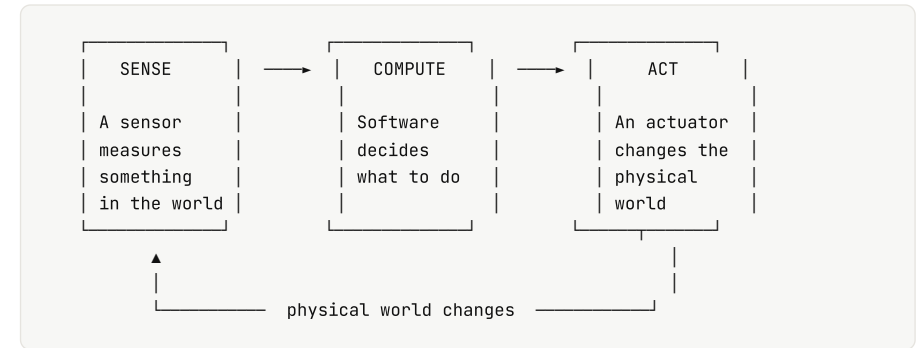
Part 1 — What a Cyber-Physical System Actually Is



Every CPS has the same anatomy: a physical layer at the top, sensors pushing readings down, a message broker fanning them out, storage and intelligence on the side, actuators pushing decisions back up.

The sense–compute–act loop

A cyber-physical system, or CPS for short, is software that reads the physical world, decides something, and changes the physical world. Then it reads the physical world again to see what changed. That cycle repeats forever.



A thermometer plus a heater plus a brain that says "turn on the heater when cold." That is the simplest CPS.

The cycle does not happen once. It happens millions of times a day. A safety-critical loop, like a fire suppression system, may run several times a second. A climate-control loop may run once every few minutes. The right speed depends on how fast the physical world can change.

What makes a CPS hard

Three things separate CPS engineering from regular software:

The physical world does not wait. A web service can take 500 milliseconds to respond and nobody minds. A fire detector that takes 500 milliseconds to react to smoke is unacceptable. The clock is the physics of the room, not the developer's patience.

The physical world has limits. A fan cannot spin faster than its motor allows. A door cannot close faster than physically possible. A temperature cannot drop by 10 degrees in one second because there is not enough cooling power. Every command must respect those limits.

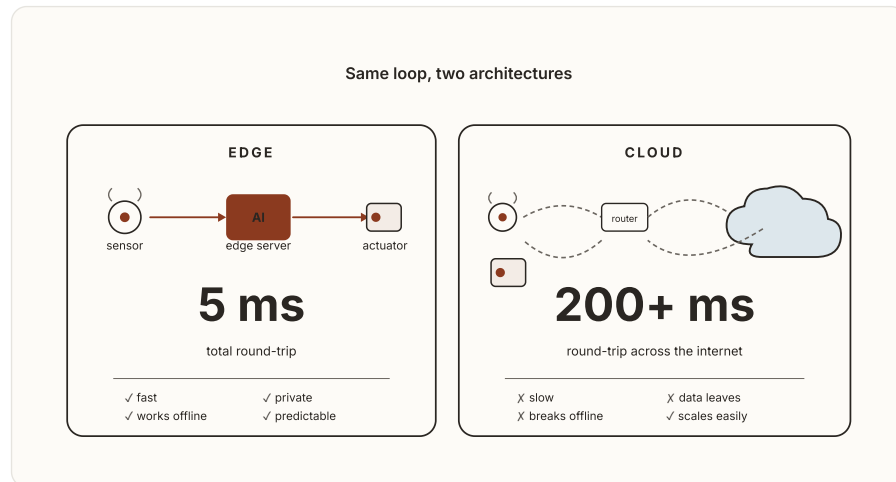
The physical world has consequences. A wrong decision in a web app means a wrong page renders. A wrong decision in a building control system can start a fire, lock people in during an emergency, or freeze a server room. Bugs cost more than money.

A building as a CPS

A modern building has thousands of sensors and actuators. The physical layer includes the rooms themselves, HVAC ducts, temperatures, smoke concentrations, lighting levels, people moving around, and air flowing through vents. The cyber layer includes the software that observes all of that and decides what to do.

The two layers meet at one specific seam: the **API of the building control system**. Sensors report through this API. Actuator commands come back through this API. In production, that API is usually a field bus protocol — BACnet, KNX, or Modbus. In this course, the same role is played by **BuildSim**, a REST and WebSocket API that simulates a real building. The architecture above the API is identical whether the bottom layer is a simulated building or a real one. Only the wiring at the very bottom changes.

Part 2 — Where the Brain Should Live: Edge vs Cloud



*Putting the same control loop in two places gives wildly different latency.
For anything safety-critical, the brain belongs on the edge.*

Many newcomers reach for "the cloud" as a default place to run software. For a CPS, that is usually the wrong default. The right place is the **edge** — a computer physically close to the building.

Why local processing matters

Four reasons drive everything to the edge.

Latency. A round trip from a building in Sweden to a Google data centre in Belgium takes roughly 50 to 200 milliseconds when everything works. When the network is congested or the internet is degraded, it can be much worse. A fire suppression system that needs to respond within one second cannot tolerate a 200 ms baseline plus a model that takes another 200 ms plus a return trip. The budget runs out before the system has even started thinking.

Bandwidth. Imagine 500 sensors in a building, each producing a 200-byte reading every five seconds. That's about 20 kilobytes per second. Trivial to send to the cloud. Now imagine each sensor is a 1080p camera at 30 frames per second. That's 500 megabytes per second per camera. Sending that to the cloud continuously is physically impossible and expensive even if it were possible. The edge processes, compresses, and filters first.

Privacy. Video feeds and occupancy patterns are sensitive personal data. Cross-border data transfers are restricted by laws like the European GDPR. Processing on-premises eliminates the risk that someone's movement patterns leave the country.

Reliability. Cloud services go down. Internet connections go down. A building that cannot control its HVAC because Amazon Web Services is having an outage is an embarrassment. The edge keeps running when the wider network does not.

The compute hierarchy

"Edge" is not a single place. It is a continuum with four tiers, each trading speed against compute power.

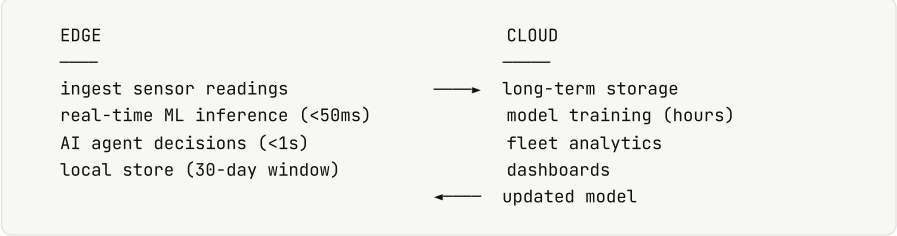
TIER	WHERE IT SITS	LATENCY	COMPUTE POWER	TYPICAL EXAMPLES
Device	On the sensor or actuator itself	<1 ms	Very low	Arduino, ESP32, Raspberry Pi
Edge	Local gateway, building server room	1–10 ms	Medium	Intel NUC, Jetson, small industrial PC
Fog	Building or campus level	10–50 ms	High	Rack server, mini data centre
Cloud	Regional data centre	50–200 ms	Effectively unlimited	AWS, Azure, Google Cloud

The terminology comes from two ideas that grew up separately. "Edge computing" was named by Shi et al. in their 2016 paper for IEEE IoT Journal. "Fog computing" was named by Cisco in 2012 and formalised by the OpenFog Consortium. For a single building, the distinction between edge and fog is mostly academic. The line gets sharper when one organisation manages many buildings.

A natural distribution in production looks like this. Simple threshold alarms and sensor firmware run at the device level. Real-time machine learning inference and agent decision-making run at the edge level. Long-term storage, model training, fleet analytics, and dashboards run in the cloud.

The hybrid pattern: train in the cloud, run at the edge

The dominant real-world architecture is hybrid. Training a machine-learning model requires lots of data and significant compute time. That work is too slow to do at the edge and benefits from cheap cloud GPUs. Once trained, the model is downloaded to edge servers where it makes predictions in tens of milliseconds. Periodically, fresh data is shipped back to the cloud, models are retrained, and the new model is pushed down to the edge.



Tools like TensorFlow Lite and ONNX Runtime exist specifically to make this pattern smooth: train in the cloud with full TensorFlow or PyTorch, convert the model to a smaller, faster format, deploy to the edge.

Compute continuum orchestration

The hybrid pattern creates a practical headache. Workloads need to run in different places — a sensor pre-processing job on a Raspberry Pi, a model-training job on a GPU cluster, a dashboard on a cloud VM. Each platform has different capabilities and different APIs. Manually deploying to each is painful.

Compute continuum orchestrators abstract this away. The developer submits a workload description that says "this job needs a GPU" or "this job needs to run at the edge." The orchestrator decides which machine fits, dispatches the workload there, and tracks it.

ColonyOS is one example, developed in Sweden and used in industrial settings. A "colony" is the orchestration boundary. **Executors** register the kinds of work they can do — GPU, edge, IoT, CPU. **Function specifications** describe what to compute, written as a small JSON document. A **broker** matches each function specification against available executors. Every message is cryptographically signed, so executors can run on untrusted infrastructure (zero-trust).

For a building control system, this enables a pattern like the following:

- Sensor data preprocessing runs on an edge executor in the building's server room.
- Anomaly model training is submitted as a function specification that the broker dispatches to a GPU executor in the cloud.
- The trained model is automatically deployed back to edge executors.
- If the GPU executor is busy, training queues or overflows to another GPU.

This is the architecture the lecture refers to as "the edge–cloud continuum." It scales beyond a single machine without forcing every component into a specific cloud.

When edge is essential vs when cloud is fine

Use edge processing when:

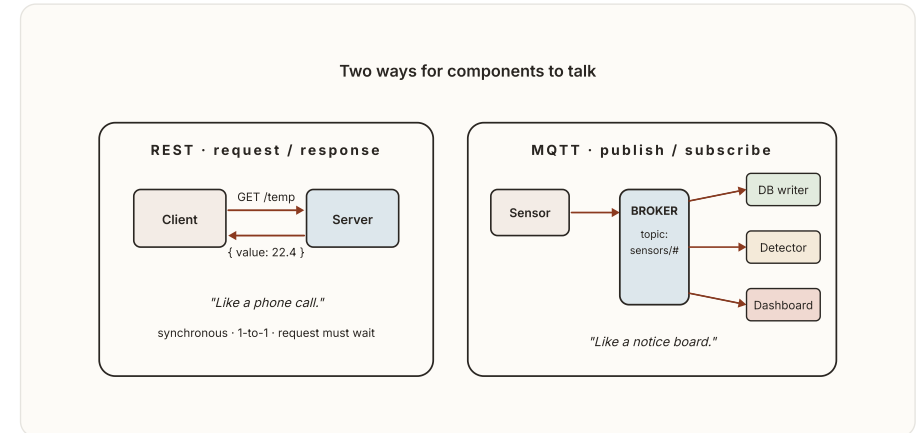
- Response time must be under one second (fire suppression, emergency unlock).
- The system must keep working without internet (safety-critical functions).
- Continuous data volume is too large to stream (video, high-frequency sensors).
- Data is sensitive and cannot leave the premises.

Use cloud processing when:

- Latency over five seconds is acceptable (dashboard updates, weekly reports).
- Compute requirements exceed local hardware (training a large model).
- Data must be aggregated across many buildings (fleet benchmarking).
- The job is rare or scheduled (disaster recovery, backup).

A common mistake is to over-edge — running everything on a single device because "it's the edge," then discovering that the device cannot run a large language model fast enough. The edge in a real building is not a Raspberry Pi. It is a rack server or a small cluster of industrial computers with serious compute. In a course-scale setup, a developer laptop plays this role.

Part 3 — How Components Talk to Each Other



REST is a phone call: synchronous, one-to-one. MQTT is a notice board: one publisher, many subscribers, none of them blocking each other.

Once components are placed at the right tier, the next question is how they communicate. Each communication pattern has different latency, scalability, coupling, and failure characteristics. Choosing the wrong one is one of the most common architectural mistakes.

Request–response (REST over HTTP)

In this pattern, one component sends a request and waits for the answer. The simplest model and the default for most web APIs.

REST stands for Representational State Transfer. It uses the verbs of HTTP — GET, POST, PUT, DELETE — to express intent. Resources are identified by URLs. Responses carry standard status codes (200 for success, 404 for not found, 500 for server error). The BuildSim API is a REST API: `GET /api/equipment` lists equipment, `POST /api/actuators/{id}/state` commands an actuator.

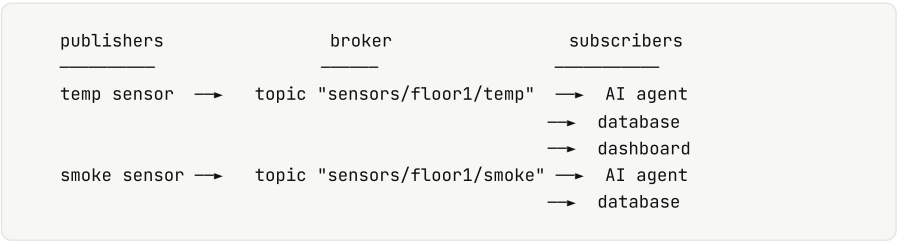
Request–response works well for control commands (set the actuator's state, then verify the state changed), one-off queries (get the current temperature), and configuration (register a new sensor). It works badly for high-frequency data streams. Polling a REST endpoint at 10 Hz consumes resources for almost nothing, since most polls return the same value. It also works

badly for fan-out, where one event needs to reach many subscribers — each subscriber would have to poll independently.

The crucial property of request–response is **tight coupling**: the client must know the server's address, the server must be reachable, and both sides block while the call is in progress. If the server is down, the client fails. There is no buffering, no retry semantics by default, no asynchrony.

Publish–subscribe (MQTT, Kafka, RabbitMQ)

A different model entirely. Publishers send messages to a **topic** without knowing who is listening. Subscribers register interest in topics and receive all matching messages. A **broker** sits in the middle, routing messages from publishers to subscribers.



The temperature sensor does not know whether one, ten, or zero consumers are listening. An AI agent can subscribe to a wildcard pattern, `sensors/+ /smoke`, and receive all smoke readings from all floors without those sensors knowing the agent exists.

This pattern offers **loose coupling**. Adding a new consumer requires no change to publishers. Subscribers can come and go. The broker buffers messages if a subscriber is briefly unavailable.

MQTT is the standard pub/sub protocol for IoT. It uses tiny message headers, just two bytes, which matters for battery-powered devices and constrained networks. It supports three quality-of-service levels: at-most-once delivery, at-least-once delivery, and exactly-once delivery. Mosquitto is the standard open-source MQTT broker.

Kafka is a different beast. It looks like pub/sub from the outside but is internally a distributed, durable, append-only log. Every message published to a topic is stored, and subscribers can replay history from any point. This makes Kafka the natural choice for analytics pipelines and event sourcing, at the cost of being heavier to operate than MQTT.

RabbitMQ sits between MQTT and Kafka. It is a general-purpose message broker that supports pub/sub, work queues, request-reply patterns, and complex routing. It speaks AMQP natively but can also speak MQTT through a plugin. Good when MQTT's simplicity is not enough but Kafka's scale is overkill.

Pub/sub is the right fit for sensor data distribution (one publisher, many consumers), system events (the building's fire alarm being triggered), and loose coupling between independently developed components.

Event-driven architecture

An **event** is an immutable record of something that already happened. Not a command ("turn on the fan") but a fact ("the temperature exceeded 28°C at 14:32:05"). Components that care about that fact react independently.

Event-driven architecture, abbreviated EDA, is the pattern where components communicate only through events. Two key properties fall out of this.

Temporal decoupling. A component that reacts to an event does not have to be running at the moment the event is produced. If the event store keeps a history, the consumer can catch up later. Systems become resilient to partial failures.

Event sourcing. Storing every event in an append-only log means the entire state of the system can be reconstructed at any point in time. If a bug is discovered, replay the log up to the moment before the bug and analyse it. This is invaluable for debugging and auditability.

For a fire alarm, event-driven architecture is natural. The single event "fire alarm triggered in A2306" causes sprinklers to activate, doors to unlock, HVAC to shut down, and an alert to be sent to the fire department. Each is a separate reaction by a separate component, all triggered by one event.

WebSocket

WebSocket provides a persistent, two-way TCP connection between a client and server. Once opened, either side can send data at any time. This is fundamentally different from HTTP, where the client must initiate every exchange.

The BuildSim API uses WebSocket to push live sensor readings and viewer state changes to the browser. The browser opens a WebSocket on page load and receives a continuous stream of

updates without polling. Polling a REST endpoint at 10 Hz to achieve the same effect would be 600 requests per minute per client, most of them returning nothing new.

WebSocket is the right fit for real-time server-to-client push (sensor streaming, live dashboards), bidirectional real-time communication, and long-lived connections. It is the wrong fit for occasional queries, where the setup cost of opening the connection outweighs the benefit, and for stateless interactions, where every request is independent.

gRPC

gRPC is a high-performance request-response protocol developed at Google. It uses HTTP/2 transport and Protocol Buffers for serialisation. The result is typed APIs with low latency and good multiplexing. It supports streaming in both directions, which lets it cover some of the WebSocket use cases too.

gRPC fits internal service-to-service communication where performance matters and both ends are controlled by the same team. It is less common in IoT, where MQTT dominates, but appears in modern microservice stacks.

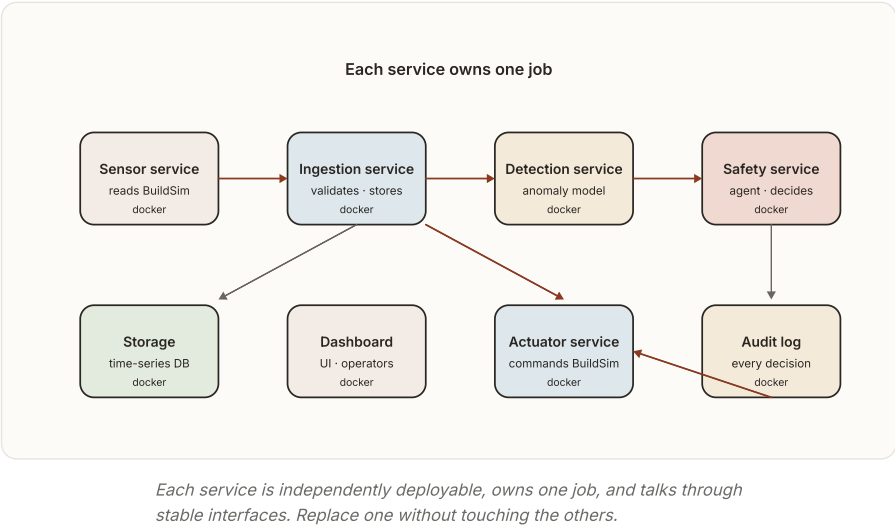
The pattern comparison table

Pattern	Coupling	Latency	Scalability	Durability	Building use
REST commands, queries	Tight	Low	Medium	No	Control
MQTT pub/sub distribution	Loose	Low	High	Optional	Sensor
RabbitMQ routing	Loose	Low	High	Yes	Complex
Kafka building log	Loose	Medium	Very high	Yes	Multi-
WebSocket streams	Medium	Very low	Medium	No	Real-time
gRPC services	Tight	Very low	High	No	Internal

The most useful rule of thumb: REST when one component asks another a specific question and waits; pub/sub when one source feeds many consumers; WebSocket when the server needs to

push to a client. Kafka enters the picture only when the system needs durable event history at high throughput.

Part 4 — How Components are Organised: Service-Oriented Architecture



The patterns above describe **how** components talk. The next question is **how big** each component should be.

Monolith versus microservices

A **monolithic** architecture puts all functionality into a single deployable unit. One process, one binary, one deploy. Simple to operate when small, painful when large.

A **microservices** architecture decomposes that single binary into small, independently deployable services that communicate over the network. Each service does one thing. Each can

be updated, scaled, and operated independently.

For a CPS like a building control system, microservices fit well for four reasons.

Different components have different scaling requirements. The data pipeline needs storage. The AI agent needs CPU or GPU. The dashboard needs nothing. Mixing them in one process means provisioning for the worst-case need everywhere.

Different components evolve at different speeds. The sensor process is stable — once it works, it rarely changes. The agent logic, by contrast, evolves rapidly as new use cases are added. Microservices let the agent be redeployed many times a week without touching the sensor process.

Different components are written by different teams or in different languages. A research team writing the ML model in Python should not be forced to write the high-frequency sensor ingestion in Python too. The sensor ingestion can be in Go for performance; the agent in Python for ML ecosystem; they talk over the network.

Failure isolation. A bug in the dashboard does not take down the safety agent. In a monolith, a memory leak in the dashboard module crashes the whole binary, including safety.

The cost of microservices is operational complexity. Multiple containers must be orchestrated. Network failures between services must be handled gracefully. Deployment configuration multiplies. For a course project, this complexity is instructive rather than a burden. It teaches real-world architecture.

Containers and Docker

A **container** packages a service together with all its dependencies — libraries, runtime, configuration files — into a single image that runs identically on any machine that has Docker installed. The "works on my laptop" problem disappears, because the laptop and the production server run the same container image.

A **Dockerfile** is the recipe for building a container image. It specifies the base operating system, the libraries to install, the application code to copy in, and the command to run at startup. Running the Dockerfile produces an **image**, a static snapshot. Running the image produces a **container**, a live instance with its own filesystem, network, and process tree.

Docker Compose is the tool for running multiple containers together. A `docker-compose.yml` file lists every container, its image, its port mappings, its dependencies on other containers, and

its environment variables. Running `docker compose up` starts all of them in the right order. The compose file is essentially the executable version of a C4 container diagram.

Each sensor and actuator as a service

In a service-oriented building control system, every sensor and every actuator is treated as a service with a well-defined interface. The architecture might include `smoke-sensor-service`, `temperature-sensor-service`, `hvac-actuator-service`, and `door-actuator-service`, each as a separate container. They share the same patterns: each publishes or subscribes on agreed topics; each registers itself with the building's API on startup; each can be killed and restarted without affecting the others.

This has two important consequences. First, the AI agent does not need to know whether the smoke sensor reads from a real device, a BuildSim simulator, a CSV file replay, or a generative ML model. As long as `sensors/smoke` carries the same JSON shape, the agent reads the same way. Second, scaling is per-service. If a building has 500 smoke sensors, you can spin up 500 instances of the same `smoke-sensor-service` image, each with different configuration.

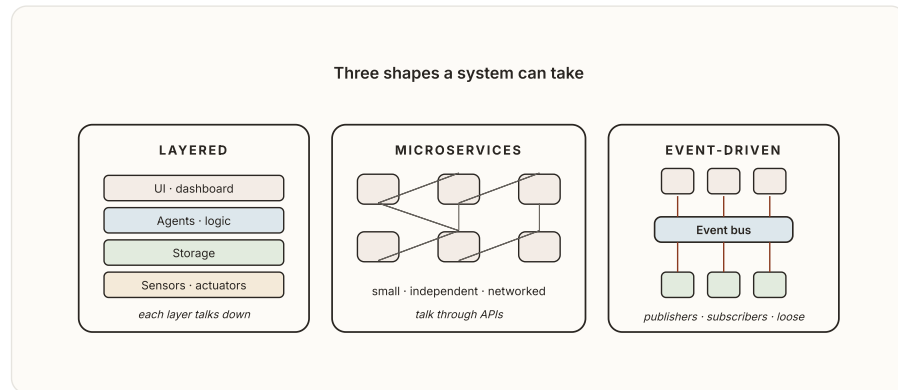
Eclipse Arrowhead

For industrial CPS, a reference architecture called the **Eclipse Arrowhead Framework** treats every sensor, actuator, and service as a registered, discoverable, and authorised service. Three core systems make this work:

- **Service Registry.** Every service registers itself; consumers look up producers by service type rather than by hardcoded address.
- **Authorisation System.** Enforces which services are allowed to talk to which. A guest sensor cannot pretend to be the building's primary smoke detector.
- **Orchestration System.** Routes requests from consumers to appropriate providers, picking the best available.

Arrowhead is used in Swedish industrial automation and is worth knowing as the production analogue of what we sketch with Docker Compose and MQTT in the course.

Part 5 — Higher-Level Architectural Patterns



Different shapes optimise for different things: layered for simplicity, microservices for scale, event-driven for loose coupling between components that come and go.

The previous parts cover individual building blocks. This part describes whole-system patterns that combine them.

Event-driven pub/sub architecture

The most common pattern for building control. Sensors publish readings to a message broker. Consumers — AI agents, databases, dashboards — subscribe to the topics they care about. The broker handles routing.

Advantages: loose coupling, easy extension (a new consumer is added without changing producers), and a natural fit for the one-to-many relationship between sensors and downstream systems. The default choice for sensor data collection, event distribution, and real-time alerting.

Lambda architecture

The Lambda architecture splits data processing into two parallel paths. The **speed layer**, also called the hot path, processes events in real time and produces low-latency approximate results — current temperature, active alerts, anomaly scores in the last minute. The **batch layer**, also called the cold path, processes historical data in bulk and produces accurate long-term results —

weekly energy reports, monthly trends, training data for the next model. A **serving layer** merges results from both paths when a query comes in.

For building control, the speed layer handles real-time ML inference and safety responses, while the batch layer trains models, generates reports, and populates the feature store for the next training run.

Lambda has been criticised because the same business logic ends up implemented twice — once in the streaming path and once in the batch path — which is hard to keep in sync. The **Kappa architecture**, proposed by Nathan Marz, simplifies this by using a single stream-processing layer for both real-time and historical processing, replaying historical data through the same code. Apache Flink supports both patterns.

Multi-agent architecture

Instead of one agent that controls everything, several specialised agents each focus on a single objective. A typical decomposition:

- A **Safety Agent** monitors smoke and fire sensors and responds to emergencies. It has the highest priority — it always overrides.
- An **Energy Agent** optimises HVAC schedules to minimise consumption.
- A **Comfort Agent** maintains temperature and air quality within occupant preferences.
- A **Coordination Layer** resolves conflicts. Safety always wins. Energy and comfort negotiate.

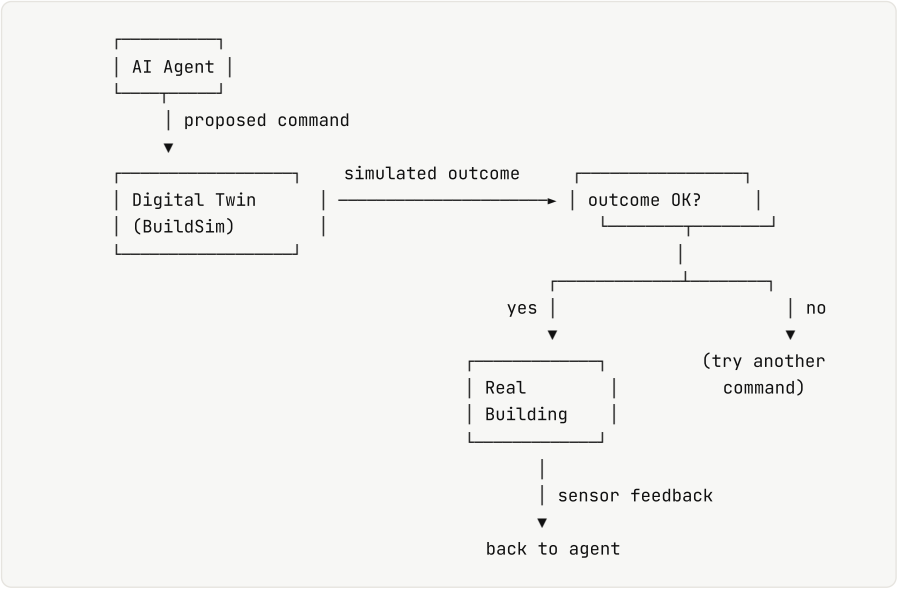
Each agent is a separate process. Each can be updated independently. The coordination layer is itself a separate component, which can be implemented as priority-based (the simplest), auction-based (each agent submits a bid), or consensus-based (agents converge through messaging). Multi-agent is the dominant pattern when the system has competing objectives.

Digital twin feedback loop

A **digital twin** is a real-time simulation of the physical system that runs in parallel with the real one. The idea is to use the twin as a sandbox: before sending a command to a real actuator, send it to the twin and observe the predicted outcome. If the twin says the outcome would be bad, the agent tries a different command.

BuildSim is itself a digital twin of a building. The agent can ask it questions like "if I turn HVAC zone 3 on, what will the temperature be in ten minutes?" The agent uses the answer to choose

better actions before committing them to the real building. For higher-fidelity simulation, tools like the Building Controls Virtual Test Bed and EnergyPlus exist in research labs.

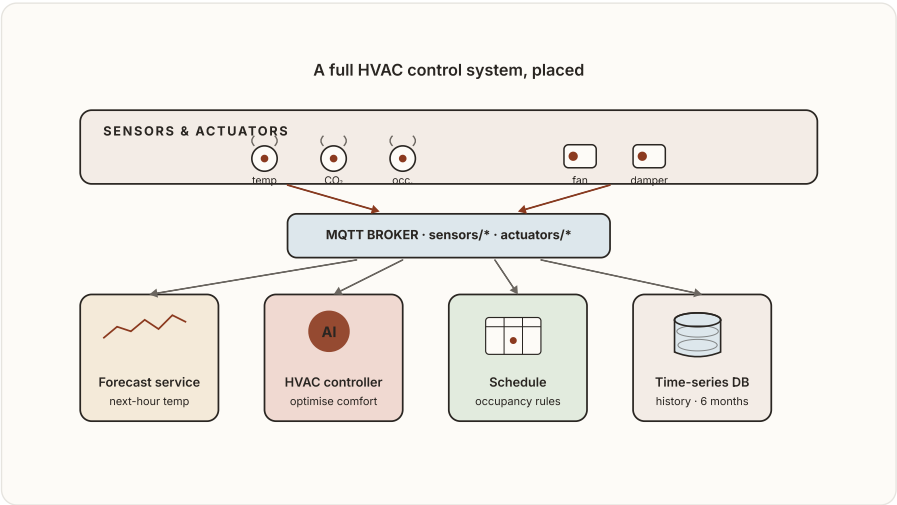


Choosing a pattern

No single pattern fits every use case. A few rules of thumb help:

- A latency requirement below 100 milliseconds points to local processing with direct in-process calls.
- A latency budget below one second is comfortable on the edge with pub/sub.
- A latency budget above one second admits the cloud.
- When many consumers need the same data, prefer pub/sub over request-response.
- When the system must keep working without connectivity, all critical logic must be at the edge with local storage and an offline-first design.
- Simple threshold decisions fit rule engines. Contextual trade-offs fit LLM agents.
- A single objective is served by a single agent. Multiple competing objectives need multi-agent coordination.

Part 6 — A Worked Example



A full HVAC control system, with each component placed and the data flowing top to bottom: physical world → broker → services → back to the world.

To make all of the above concrete, consider a fire detection system and walk through the architectural choices.

The use case: smoke sensors and temperature sensors are deployed in every room. When smoke is detected, the system must activate sprinklers in the affected room within one second, unlock fire doors on the evacuation path, and notify the building manager. False positives must be kept under 5 percent.

Where does the brain run? At the edge. The one-second latency budget rules out cloud reasoning on the critical path.

How do components communicate?

- Sensor readings: pub/sub via MQTT. Many consumers want the same readings (the database, the anomaly detector, the dashboard), and the natural fan-out makes MQTT a clean fit. Topics use a hierarchy: `sensors/Level0/A2306/smoke`.

- Actuator commands: REST. One agent sends one command to one actuator and wants to know the result.
- Browser updates: WebSocket. The dashboard needs live updates without polling.

How are components organised? As microservices in Docker containers.

- One container per sensor type (`smoke-sensor-service` , `temp-sensor-service`).
- One container per actuator type (`sprinkler-service` , `door-service`).
- One MQTT broker container.
- One time-series database container.
- One anomaly-detection container (the ML model, edge-deployed).
- One safety-agent container (the LLM-driven reasoning, edge-deployed but calling out to the cloud LLM when needed).

Which higher-level pattern? Event-driven pub/sub for the data flow, with multi-agent on the decision side. A safety agent reacts to anomaly events. An energy agent monitors HVAC schedules. A coordination layer resolves conflicts. A digital twin (BuildSim) lets the agent test a sprinkler command in simulation before committing.

Where does training happen? In the cloud, on a GPU server, scheduled overnight. The trained anomaly model is downloaded to the edge and replaces the previous model on the next agent restart.

What is the deployment plan? A `docker-compose.yml` on the building's edge server brings up all containers. A ColonyOS executor running on the same server picks up nightly training jobs from a colony broker. The GPU server runs another executor that takes the heavy training jobs.

That sequence of choices — edge for inference, MQTT for sensor fan-out, REST for commands, microservices for failure isolation, multi-agent for competing objectives, digital twin for safety, ColonyOS for training orchestration — is the architecture this lecture trains its readers to design.

Part 7 — Vocabulary Reference

Every term used in this chapter, defined.

TERM	DEFINITION
CPS (Cyber-Physical System)	A system where software directly senses and changes the physical world in a closed loop
Sense-compute-act loop	The continuous cycle: read sensor, decide action, command actuator, observe effect
Edge computing	Running software on hardware physically close to where the data is generated, instead of in a remote data centre
Fog computing	A distributed layer of compute spanning edge and cloud, typically at building or campus scale
Compute continuum	The unified abstraction across device, edge, fog, and cloud resources
REST	A request–response API style using HTTP verbs (GET, POST, PUT, DELETE) and JSON over HTTP
MQTT	A lightweight publish/subscribe protocol designed for IoT, using topics and a broker
Kafka	A distributed durable event log that looks like pub/sub from the outside but stores messages for replay
RabbitMQ	A general-purpose message broker that supports many messaging patterns through the AMQP protocol
WebSocket	A persistent bidirectional connection that allows the server to push data to the client
gRPC	A high-performance typed RPC framework using HTTP/2 and Protocol Buffers
Event-driven architecture	A pattern where components communicate only through immutable event records, never direct commands
Event sourcing	Storing every event in an append-only log, enabling full state reconstruction at any past moment

TERM	DEFINITION
Monolith	A single deployable unit containing all of an application's functionality
Microservice	A small independently deployable service that does one thing and communicates with others over the network
Container	An isolated runtime that packages a service with all its dependencies; the standard packaging for microservices
Docker	The most common implementation of containers, with a build format (Dockerfile) and runtime
Docker Compose	A tool for defining and running multi-container applications, configured by a YAML file
Eclipse Arrowhead	A service-oriented framework for industrial IoT, with built-in service registry, authorisation, and orchestration
ColonyOS	A meta-orchestrator that dispatches workloads across edge, GPU, and cloud executors based on resource requirements
Lambda architecture	A pattern that splits data processing into a real-time hot path and a historical batch cold path
Kappa architecture	A simplification of Lambda using a single stream-processing layer for both real-time and historical data
Multi-agent system	An architecture with several specialised AI agents that coordinate through priority, auction, or consensus
Digital twin	A real-time simulation running in parallel with the physical system, used to predict outcomes of proposed actions

Part 8 — Summary in Five Sentences

1. A cyber-physical system is software that senses the physical world, decides, acts on it, and observes the result, in a continuous loop fast enough that the physics cannot run away from it.
2. The brain belongs at the edge, close to the sensors, because latency, bandwidth, privacy, and reliability all push against putting it in the cloud.
3. Communication patterns differ in coupling and durability — pick REST for control commands, pub/sub for sensor data, WebSocket for server-push to browsers, Kafka when history matters.
4. Components are small, single-purpose, independently deployable services in containers, communicating over well-defined interfaces, with no shared state.
5. Higher-level patterns combine these building blocks — event-driven pub/sub for data flow, Lambda or Kappa for the speed/batch split, multi-agent for competing objectives, digital twin for safety-critical command verification.

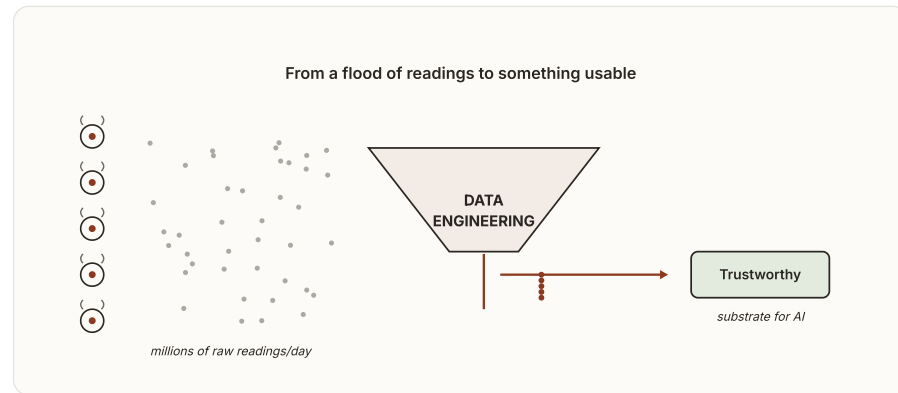
These five ideas, properly internalised, are the foundation for every architectural decision in the rest of the course.

Lecture 3 — Data Engineering for Cyber-Physical Systems

Standalone notes for the third lecture of D7065E. Read on its own or alongside

`lectures/lecture-3-data-engineering.md`.

Part 1 — Why a Whole Lecture About Data



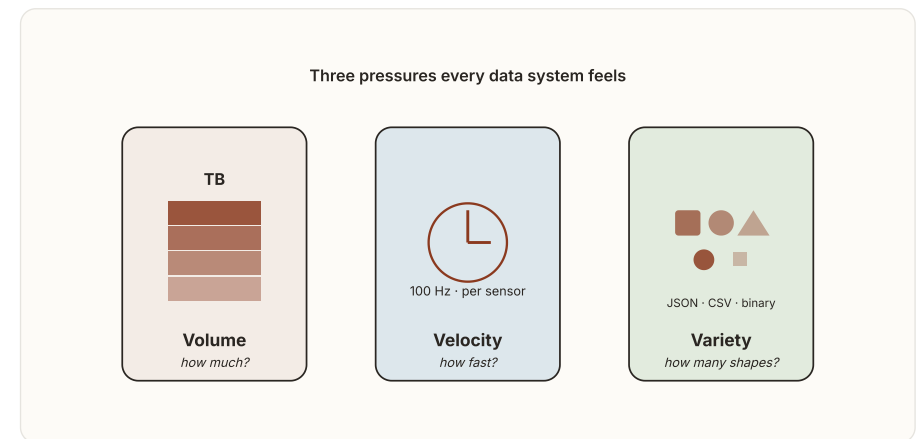
A modern building generates millions of readings a day. Data engineering is the discipline that turns that flood into a trustworthy substrate an AI can act on.

Before any AI agent can be smart, it needs data. Lots of it, well-organised, fresh enough to be useful, and clean enough to trust. The plumbing that gets raw sensor readings from the building into the agent's hands is called **data engineering**, and it is just as important as the agent itself. A brilliant model trained on bad data will produce bad predictions. A simple model trained on excellent data often beats a sophisticated model trained on garbage.

A useful analogy: think of the AI agent as a chef and the data pipeline as the kitchen staff who source ingredients, wash them, chop them, and lay them out on the counter. The chef gets the credit when the dish is good, but the dish was never going to be good if the ingredients arrived rotten, in the wrong portions, an hour late.

This chapter is the kitchen.

Part 2 — The Three Pressures: Volume, Velocity, Variety



Volume, velocity and variety are the three pressures every data system has to absorb. They shape every choice that follows.

A modest commercial building with 100 sensors, each reporting every 5 seconds, produces about 1.7 million readings per day. Multiply that across a year and the system holds half a billion data points. Add video cameras, vibration sensors at industrial frequencies, and access events, and the number reaches hundreds of millions of records per month.

The volume itself is not the hard part. The hard part is that data engineering for a cyber-physical system has to satisfy three demands at once, and the demands pull in different directions.

Volume

How much data, total, accumulated over time. A weather station for one room is small. A building with thousands of sensors over several years is large. The architecture must store this

without running out of disk space and without slowing down to a crawl.

A useful image: a single garden hose versus a fire hose versus a river. The amount of water you can collect determines whether you need a bucket, a tank, or a reservoir.

Velocity

How fast new data arrives, and how fast decisions need to be made on it. A safety system that detects fire must process incoming readings in milliseconds. A weekly energy report can take its time. Both are valid, but they need different infrastructure.

A useful image: drinking a glass of water versus drinking from a fire hose. Same liquid, very different consequences.

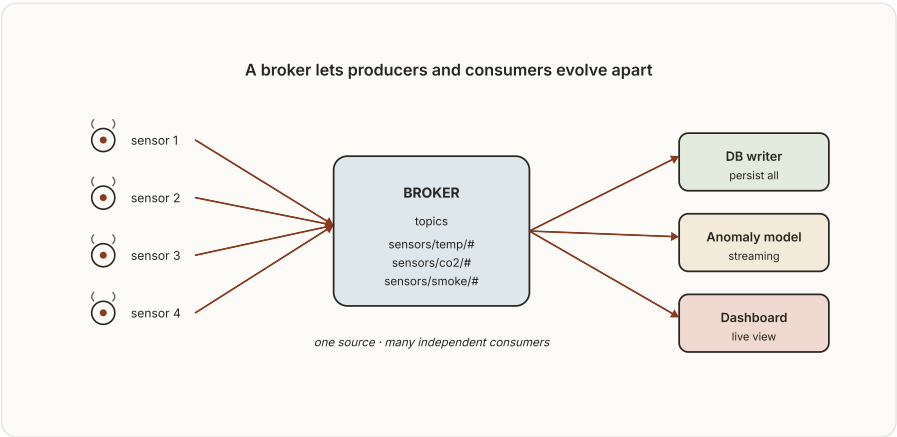
Variety

How many different shapes the data takes. Numeric readings, boolean states (door open/closed), strings (event types), images, time series. A pipeline that handles only one shape is brittle.

A useful image: a smoothie maker that takes apples, ice, yoghurt, and spinach — all of which need to go through different processing before they can be combined.

The fundamental tension: real-time decisions need data within seconds, machine-learning training needs months of clean labelled data, and long-term analytics needs years of queryable history. These three goals require different storage systems, different processing patterns, and different data representations. A well-designed pipeline serves all three without compromise.

Part 3 — Getting Sensor Data Into the System (Ingestion Patterns)

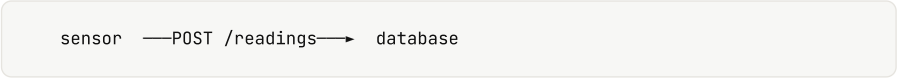


A broker in the middle decouples sensor producers from the consumers downstream. New consumers can subscribe without anyone changing the sensors.

The first step in any pipeline is **ingestion**: getting data from sensors into a durable store. Three patterns dominate, each fitting a different scale.

Pattern A: Direct REST POST

Each sensor sends every reading directly to an HTTP endpoint that writes it into a database immediately.



Easy to set up. Works fine at low volume. Breaks at scale: 100 sensors at 10 Hz means a thousand HTTP requests per second, which overwhelms most databases. There is also no buffer — if the database is slow or temporarily down, readings are lost.

Analogy: handing every letter directly to the postman, who must be standing on your doorstep every time you want to send one. Fine if you write one letter a week. Disastrous if you run a mail-order business.

Pattern B: Message queue as buffer

Each sensor publishes to a message broker. A separate consumer process reads from the broker and writes into the database.



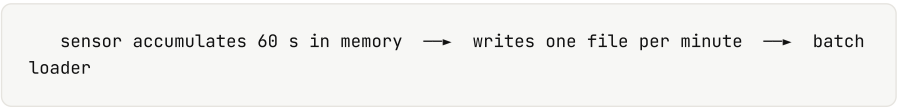
The broker acts as a shock absorber. If the database is briefly slow, messages accumulate in the broker, and the consumer catches up later. The sensor doesn't care whether the database is up.

This is the recommended pattern for the course: MQTT (using Mosquitto as the broker) plus a Python consumer that writes into TimescaleDB or DuckDB.

Analogy: the postman is replaced by a mailbox in the village square. You drop your letter in any time, the postman collects them on a schedule, the receiving end opens its mailbox when ready. Three actors, each independent.

Pattern C: Batch file upload

The sensor accumulates readings in memory and writes them to a file (typically a Parquet file) once a minute or once an hour. A separate process reads those files into a database.

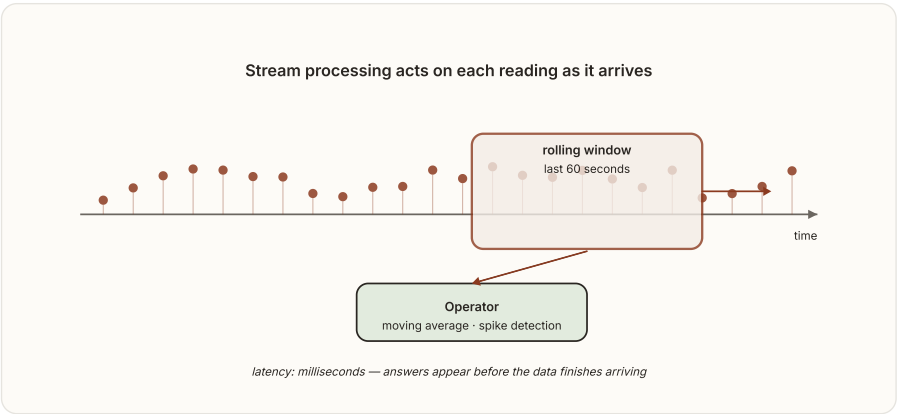


Highest throughput, lowest network overhead. Bad for real-time decisions because of the latency, excellent for historical training data.

Analogy: instead of sending one letter at a time, you wait until end-of-day and send a single envelope containing all your letters from that day. Cheap and efficient when nothing is urgent. Hopeless when something is.

Most real systems use **two patterns at once**: the broker for the real-time path, batch files for the historical path. The same reading is written into both places, and each place serves a different audience.

Part 4 — Stream Processing: Working on Data as It Flows



Stream processing keeps a sliding window of recent readings in memory and produces answers continuously, with millisecond latency.

Once data is flowing in, the next question is how to process it. Two main modes exist: streaming (processing each reading as it arrives) and batch (processing a large pile of stored data at once). This part covers streaming; the next covers batch.

A **stream** is an unbounded, time-ordered sequence of events. New events keep arriving forever. Stream processors apply operations to streams and produce either new streams or side effects (database writes, alerts, dashboard updates).

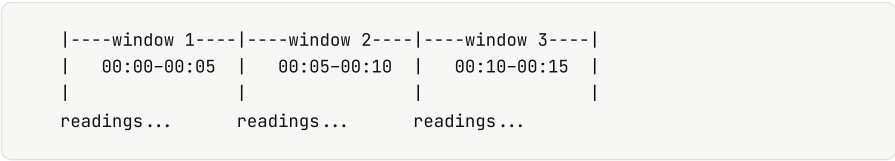
A useful image: a river flowing past a water-quality station. The station measures every drop as it goes past — there is no way to stop the river and inspect it all at once.

Windowed aggregation

A single sensor reading is rarely useful by itself. It is noisy, missing context, and easily misleading. The fix is to compute statistics over a **window** of recent readings.

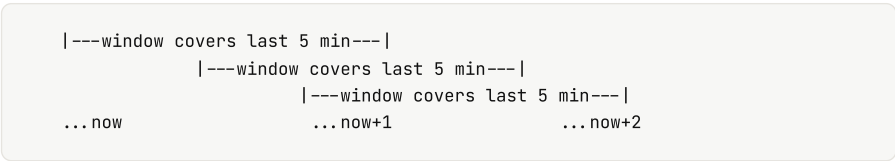
Three common window types:

Tumbling windows are non-overlapping, fixed-size buckets. Each reading falls into exactly one window.



Tumbling windows are useful for periodic reports: a 5-minute average that resets every 5 minutes.

Sliding windows are overlapping. Each new reading falls into multiple windows.



Sliding windows are useful for continuous tracking: "the rolling 5-minute average, updated every second."

Session windows are dynamic. They grow as long as events keep arriving and close when there's a gap.



Session windows are useful for grouping bursts of activity: "all access events for one person in one trip through the building."

A natural image: imagine watching a movie versus watching the news.

- Tumbling windows are like episodes — each one starts at a fixed time and runs a fixed length.
- Sliding windows are like a live ticker that always shows "the last 5 minutes."
- Session windows are like a phone call — they start when you pick up, end when you hang up, and have no fixed length.

Complex event processing

Some patterns need to span multiple events over time. A fire detection rule might say: smoke above 0.7 for ten consecutive readings, combined with temperature rising at more than 2°C per minute, combined with a door opening in the same zone recently. No single reading triggers the rule; only the pattern across many readings does.

This is **complex event processing**, abbreviated CEP. Apache Flink has a CEP library that lets you describe these temporal patterns declaratively. For smaller systems, a stateful Python process maintaining a sliding window in memory is sufficient.

Analogy: a single yawn doesn't mean someone is bored. A yawn, followed by checking their phone, followed by looking at the exit, repeated three times in five minutes — that's a pattern. CEP is the skill of recognising the pattern, not the individual symptoms.

Tools for stream processing

The right tool depends on scale.

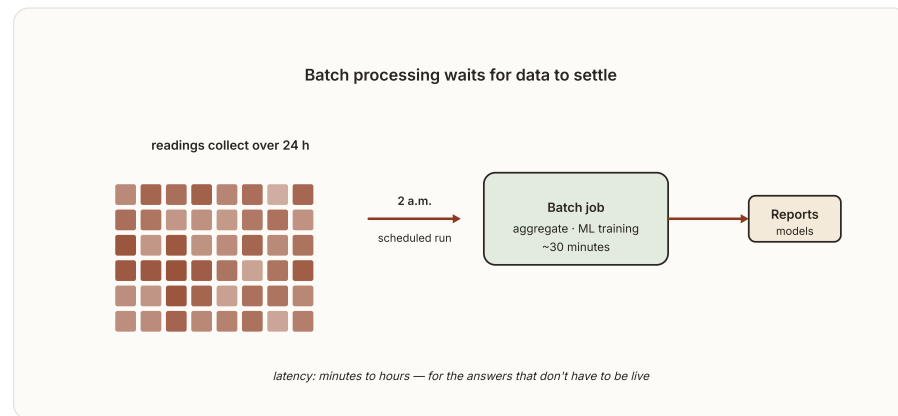
Apache Flink is a distributed stream processing engine. Production-grade and feature-rich, but heavy to operate. Right when handling millions of events per second.

Kafka Streams is a stream processing library built directly on top of Apache Kafka. If Kafka is already in the architecture, Kafka Streams is the natural choice.

Redis Streams is a lightweight stream storage feature inside Redis. Built-in consumer groups, in-memory speed, easy to run.

Plain Python with asyncio is sufficient for the course. A simple consumer that reads from MQTT, computes windowed aggregations in memory, and writes to a database is entirely fine for one building.

Part 5 — Batch Processing: Working on Data After It Settles



Batch processing waits for data to settle, then processes large windows at once. Cheaper, simpler, but the freshest answer is from yesterday.

Batch processing operates on large volumes of stored data all at once. A daily job that prepares training data, a weekly report that summarises energy use, a monthly anomaly analysis. Batch jobs run on a schedule (nightly, weekly) or on demand.

A useful image: streaming is fishing with a line — one fish at a time, you react as each one bites. Batch is fishing with a net — you collect a lot at once and process them together.

For building control, the critical batch job is **training data preparation**: pulling ninety days of sensor readings out of the data lake, computing the features the ML model needs (rolling averages, occupancy patterns, time-of-day encodings), and producing a clean Parquet or CSV file for training. This job may take minutes to hours, which is fine because real-time isn't required.

Tools for batch processing

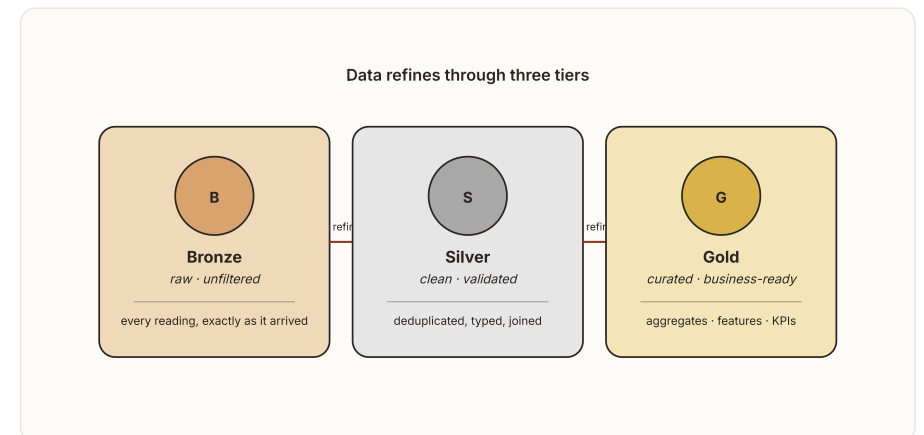
DuckDB is the recommended tool for the course. It is an in-process SQL engine that queries Parquet files directly without a server. Fast, zero-configuration, full SQL. It treats a folder of Parquet files like a database table.

```
-- DuckDB reading directly from a folder of Parquet files
SELECT room, AVG(value) AS avg_temp
FROM read_parquet('data/bronze/temperature/*.parquet')
WHERE ts > now() - INTERVAL '24 hours'
GROUP BY room;
```

pandas is a Python DataFrame library. Flexible, interactive, great for prototyping. Limited to a single machine and single thread, so it doesn't scale to industrial volumes, but for a building it is more than enough.

Apache Spark is a distributed batch processing engine. The right tool when one machine isn't enough — multi-building analytics at industrial scale.

Part 6 — The Data Lake (with the Medallion Architecture)



Bronze, silver, gold. Data gets more refined and more trustworthy at each tier — and the rules that produce each tier are versioned in code.

So far the discussion has been about *moving* data and *processing* data. The next question is where to *store* it for the long term.

The data lake philosophy

A **data lake** is a storage system that keeps raw data in its native format, exactly as it arrived, until somebody needs it. The philosophy is "store first, structure later." Unlike a data warehouse, which requires the schema to be decided up-front, a data lake lets the schema be decided at query time.

Why does this matter? Because in a real CPS, you don't know in week one that the 5-minute variance of CO2 readings will turn out to be a useful occupancy feature. By the time you discover it, you'd be furious if you had discarded that level of detail. The data lake keeps everything, in case you need it later.

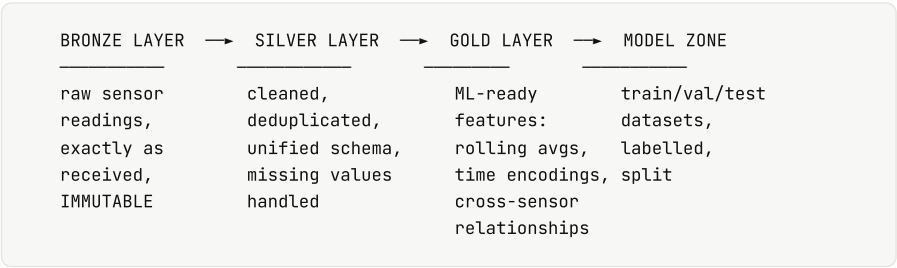
The three core principles:

- 1. **Store raw data.** Never transform or discard the original sensor reading. Always keep the immutable raw record.
- 2. **Transform on read.** Apply cleaning, feature engineering, and aggregation at query time, not at write time. This way, if you find a bug in your transformation logic, you can fix it without re-collecting the data.
- 3. **Schema on read.** Define the data's shape when querying, not when storing. This accommodates evolving schemas without painful migrations.

Analogy: a data lake is like a pantry that stores raw ingredients — flour, eggs, vegetables — exactly as they arrived from the grocery store. A data warehouse is like a freezer full of pre-cooked meals: faster to serve, but if you decide tomorrow that you want to make a salad instead of lasagna, you're out of luck.

The medallion architecture

Storing everything raw is helpful, but querying everything raw every time is expensive. The compromise is the **medallion architecture**, popularised by Databricks: organise the data lake into layers, each transformed a bit more than the previous one.



Bronze is the raw zone. Each reading is stored exactly as it came off the wire, timestamped on arrival, and never modified. If a bug is found in a downstream pipeline, reprocessing from bronze is always possible.

Silver is the cleaned zone. Validated data (out-of-range values flagged), deduplicated (retransmissions removed), unified schema across sensors, missing values handled or marked. Silver is what most queries hit.

Gold is the features zone. Business-ready features for ML: rolling statistics, derived signals, time-of-day encodings, cross-sensor relationships. This is the input to model training.

Model zone holds ready-to-train datasets with labels and train/validation/test splits.

Analogy: bronze is your shopping receipts in a shoebox. Silver is the same receipts entered into a spreadsheet with consistent columns. Gold is a monthly summary showing categories, totals, and trends. Each is more useful than the last, but each was derived from the original receipts. If you discover an error in the gold report, you can always go back to the shoebox and reprocess.

Part 7 — Storage Formats: How the Bytes Are Laid Out

A single number is just a number. A million numbers in a file is a layout decision. The chosen format affects how fast queries run, how much disk space is consumed, and how robust the data is to schema changes.

CSV (comma-separated values)

The simplest format. One row per record, fields separated by commas, first row often contains column names.

```
ts,sensor_id,room,value
2026-04-27T13:42:00,smoke-A2306,A2306,0.82
2026-04-27T13:42:05,smoke-A2306,A2306,0.85
```

Universally readable, human-friendly, supported everywhere. But slow for large data, no schema enforcement, and stored as text (which means a 0.82 takes four bytes, not the eight bytes a double-precision float would take in binary).

Good for small exports and manual inspection. Bad for production pipelines.

Parquet

The standard format for data lakes. Columnar binary, compressed, schema-enforced.

The key insight is **columnar storage**. CSV stores data row by row:

```
row 1: ts=..., sensor_id=..., room=..., value=...
row 2: ts=..., sensor_id=..., room=..., value=...
row 3: ts=..., sensor_id=..., room=..., value=...
```

Parquet stores it column by column:

```
column ts:      [ts1, ts2, ts3, ts4, ts5, ts6, ts7, ts8, ...]
column sensor_id: [s1,  s1,  s1,  s2,  s2,  s2,  s3,  s3,  ...]
column room:     [r1,  r1,  r1,  r1,  r1,  r1,  r2,  r2,  ...]
column value:    [v1,  v2,  v3,  v4,  v5,  v6,  v7,  v8,  ...]
```

This matters for two reasons. First, queries that need only the `value` column don't have to read the rest. Second, columnar data compresses much better than row-based data, because consecutive values are similar (e.g., the `room` column has just a few distinct strings repeated millions of times).

Parquet files are typically 4 to 10 times smaller than equivalent CSV files, and queries that touch only a few columns run an order of magnitude faster.

Analogy: a CSV is a stack of paper forms. Each form has multiple fields, and if you want to compute the average of one field across a thousand forms, you have to flip through every form. Parquet is a spreadsheet with one column per variable; computing the average of a column is one operation on one column.

JSON Lines (JSONL)

One JSON object per line.

```
{"ts":"2026-04-27T13:42:00","sensor_id":"smoke-A2306","value":0.82}
{"ts":"2026-04-27T13:42:05","sensor_id":"smoke-A2306","value":0.85}
```

Flexible schema, human-readable, easy to append. Useful for event logs and audit trails where new fields are added over time.

ORC

Similar to Parquet, used in Hadoop ecosystems. Parquet is preferred unless an existing Hive/Spark environment requires ORC.

Part 8 — A Self-Hosted Data Lake with MinIO and DuckDB

For the course, an entire data lake fits on a developer's laptop using two Docker containers.

MinIO is an open-source object storage system that speaks the Amazon S3 API. From any S3 client's perspective, MinIO looks identical to S3, but it runs locally. Storage organisation:

```
bucket: building-data
├─ bronze/
│   └─ temperature/
│       ├── date=2026-04-27/readings.parquet
│       └─ date=2026-04-28/readings.parquet
│   └─ smoke/
│       └─ date=2026-04-27/readings.parquet
├─ silver/
└─ gold/
```

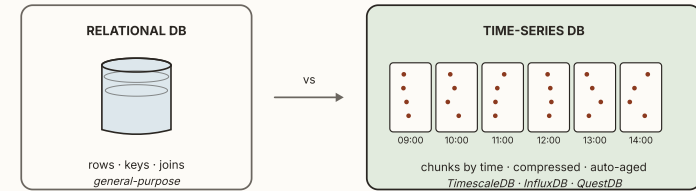
DuckDB queries this directly using the `httpfs` extension. No ETL server, no cluster, no managed service.

```
-- Query: peak smoke per sensor in the last 7 days
SELECT sensor_id,
       MAX(value) AS peak_smoke
FROM read_parquet('s3://building-data/bronze/smoke/date=*/readings.parquet')
WHERE ts > NOW() - INTERVAL '7 days'
GROUP BY sensor_id
ORDER BY peak_smoke DESC;
```

The recommended pattern for the course is to combine MinIO (the data lake) with TimescaleDB (the real-time store) and DuckDB (the SQL engine on top of MinIO). All three run in Docker. Together they cover the hot path (TimescaleDB), the cold path (MinIO + Parquet), and the analytical path (DuckDB).

Part 9 — Time-Series Databases

Time-series databases are tuned for one shape of data



Time-series databases store rows indexed by time, compressed per chunk, and discarded automatically after a retention period. Far faster than a general-purpose database for this shape of data.

A general-purpose database like PostgreSQL or MySQL is built for a typical web-application workload: lots of lookups by primary key, joins between tables, transactional updates. Sensor data has a completely different shape.

Why time-series data is different

Three properties of sensor data that don't fit a general database:

1. **Append-heavy.** Every write is a new row. Existing rows are never updated. A general database's update machinery is unused overhead.
2. **Time-range queries dominate.** Almost every question is "what happened between time A and time B?" Joining tables by primary key is rare.
3. **Downsampling.** Two-year-old 5-second data is rarely needed at full resolution. Hourly averages would do, freeing 99% of the storage.

A **time-series database**, abbreviated TSDB, is built for exactly this shape. Data is stored in time-ordered chunks. New data appends to the latest chunk. Time-range predicates immediately prune entire chunks outside the range. Retention policies automatically downsample or delete old data.

Analogy: a general-purpose database is like a filing cabinet where every paper is filed by topic. Finding "all papers about Project X" is fast; finding "all papers from last March" requires walking through every folder. A time-series database is like a diary, where every page is dated and finding "what happened in March" is just opening to March.

TSDB options

InfluxDB is a purpose-built time-series database. Its own data model (measurements, tags, fields) and its own query languages (InfluxQL, Flux). Excellent built-in tooling — the Telegraf agent for automatic sensor ingestion, native Grafana integration. Industry-standard for IoT.

TimescaleDB is a PostgreSQL extension. It adds time-series capabilities to standard PostgreSQL — data lives in regular Postgres tables, queries use standard SQL, every PostgreSQL driver works. **Hypertables** automatically partition data by time under the hood. **Continuous aggregates** keep time-windowed summaries materialised so common queries don't have to recompute them.

For the course, TimescaleDB is the most pragmatic choice: standard SQL, easy to integrate with any Python script, and it runs in one Docker container.

```
-- TimescaleDB: create a hypertable
CREATE TABLE readings (
    time      TIMESTAMPTZ NOT NULL,
    sensor_id TEXT NOT NULL,
    value     DOUBLE PRECISION NOT NULL,
    unit      TEXT
);
SELECT create_hypertable('readings', 'time');

-- Insert one reading
INSERT INTO readings (time, sensor_id, value, unit)
VALUES (NOW(), 'smoke-A2306', 0.82, 'normalised');

-- Query: 5-minute averages over the last 24 hours
SELECT time_bucket('5 minutes', time) AS bucket,
       AVG(value) AS avg_value,
       MAX(value) AS max_value
FROM readings
WHERE sensor_id = 'smoke-A2306'
  AND time > NOW() - INTERVAL '24 hours'
GROUP BY bucket
ORDER BY bucket;
```

ClickHouse is a column-oriented analytical database, extremely fast for read-heavy analytical queries. Used in production at Cloudflare and Uber. Appropriate when analytics push beyond what TimescaleDB can comfortably handle.

DuckDB on Parquet is the lightest option for historical analytics. No server, runs in-process, full SQL, reads Parquet files directly. Right for the cold path; less suited to high-frequency ingestion.

The recommended setup for the lab is **TimescaleDB for the hot (real-time) path and DuckDB/MinIO for the cold (historical) path**. Both use SQL. Both run in Docker.

Part 10 — ETL: Extract, Transform, Load

Once data is stored, getting it into a useful shape requires a small pipeline of its own. The classic name is ETL — extract, transform, load.

Extract

Reading data from one or more sources. The time-series database for recent data, the data lake for historical data, external services for context (a weather API for outdoor temperature, a calendar API for occupancy schedules).

Transform

The step where raw data becomes useful. This is also the step that students underestimate the most.

Raw sensor readings — a float every five seconds — are not informative on their own. They are noisy, high-dimensional, and lack context. Feature engineering converts them into signals that carry meaning:

RAW DATA	DERIVED FEATURE	WHY IT HELPS
Temperature readings	5-minute rolling average	Removes noise, reveals trend
Temperature readings	Rate of change (°C per minute)	Detects rapid heating (fire signature)
Smoke readings	Count of readings > 0.5 in last 5 minutes	More robust than a single reading
CO2 readings	Similarity to daily cycle	Detects occupancy anomalies
Door events	Time between events	Detects unusual access patterns
HVAC + temperature	Residual (actual – expected)	Detects HVAC failure

A useful image: feature engineering is what a chef does to crude ingredients before serving. The raw potato is not edible. Wash, peel, slice, and bake it, and now you have something useful.

Temporal features

A surprising amount of building behaviour is predictable from the clock alone. Offices fill at 8 a.m. and empty at 6 p.m. on weekdays. CO2 rises and falls in a daily cycle. A model that knows the time of day can predict half the relevant variables before looking at any sensor.

But raw hour-and-minute values confuse machine-learning models. The model sees 23:59 and 00:01 as far apart, even though they're adjacent. The fix is **cyclical encoding** with sine and cosine.

```
import numpy as np

def add_time_features(df):
    """Add cyclical time encodings."""
    t = df['timestamp']
    df['hour_sin'] = np.sin(2 * np.pi * t.dt.hour / 24)
    df['hour_cos'] = np.cos(2 * np.pi * t.dt.hour / 24)
    df['dow_sin'] = np.sin(2 * np.pi * t.dt.dayofweek / 7)
    df['dow_cos'] = np.cos(2 * np.pi * t.dt.dayofweek / 7)
    return df
```

The sine and cosine pair together place every hour on a unit circle. 23:59 and 00:01 end up close together on the circle, as they should.

Analogy: imagine writing the hours on a clock face. The model that uses the *clock face position* (which is what sin/cos gives it) understands that 11 o'clock and 1 o'clock are nearby. The model that uses *just the number* sees 11 and 1 as nine hours apart.

Cross-sensor features

Relationships between sensors are often more informative than any single sensor. A few examples:

- Temperature difference between adjacent rooms — detects a door left open or an HVAC imbalance.
- CO2 combined with ventilation state — estimates occupancy without an occupancy sensor.
- Smoke level combined with temperature gradient — distinguishes cooking (high smoke, modest temperature rise) from a real fire (high smoke, fast temperature rise).

Load

Writing the computed features to a feature store — a database or Parquet file ready for ML training — or feeding them directly into the model.

ETL tools

dbt (data build tool) is a popular open-source tool that lets transformations be expressed as SQL queries. Dbt runs queries in the correct order, tests their outputs, and generates documentation. Free, well-supported, with a free 4-hour fundamentals course.

pandas is the standard Python DataFrame library. Flexible and interactive, ideal for exploratory work. Single-threaded and in-memory, so it doesn't scale to industrial volumes; production pipelines are usually rewritten in SQL (using dbt and DuckDB) once the transformations are stable.

Apache Airflow is a workflow orchestration platform. ETL jobs are defined as directed acyclic graphs of tasks, scheduled, retried on failure, and alerted on errors. Production-grade. For a course-scale system, a Python script scheduled with cron is sufficient; Airflow is the heavy-machinery version.

Part 11 — Data Quality and Observability

Data quality is the silent killer of machine-learning systems. A model trained on bad data will produce bad predictions, and the failures are often subtle. The model performs well on the training data (because the training data is also bad), then fails mysteriously in production.

For a CPS, bad data is not just an ML problem. It is a safety problem. A smoke sensor stuck at zero will silently prevent the fire-detection system from ever responding.

Five common data-quality failures

Missing data. A sensor goes offline because the network drops, the power fails, or the sensor itself dies. The pipeline receives no readings for some period. Handling options:

- *Forward-fill:* use the last known value. Reasonable for slowly changing quantities like temperature. Dangerous for fast-changing ones like door state.
- *Linear interpolation:* estimate values based on readings before and after the gap. Reasonable for smooth signals.
- *Mark as missing:* insert a null marker that the ML model handles explicitly.
- *Alert on absence:* if a safety sensor has not reported in 30 seconds, raise an alarm. Don't fill silently — react.

Duplicate data. MQTT's at-least-once delivery means a message may be delivered twice. Without protection, every duplicated smoke reading inflates the model's view of how often anomalies happen. The fix is idempotent inserts (write only if not already present) or dedup logic in the consumer that keys on `(sensor_id, ts)`.

Analogy: imagine if every time the postman wasn't sure whether a letter was delivered, he delivered it twice. Soon your filing cabinet has two of every letter. You need a rule: if the same letter arrives twice, throw the second one away.

Stale data. The sensor process is alive, the network is fine, but the value never changes. A frozen sensor reads the same value indefinitely. Detection: compute the standard deviation of readings over a 5-minute window — if it's exactly zero, the sensor is probably stuck. Monitoring: track each sensor's last-updated timestamp and alert if nothing has changed for N seconds.

Clock drift. Each sensor has its own clock, and clocks drift. A reading from sensor A timestamped 14:32:00 and one from sensor B timestamped 14:31:58 may have happened in the opposite order from what the timestamps suggest. Mitigations: run NTP (Network Time Protocol) on every device, store both the device timestamp and the time when the server received the message, and prefer the server timestamp for ordering.

Analogy: imagine a courtroom where every witness uses a different clock. Their statements about timing can't be compared without first synchronising the clocks.

Schema evolution. A new sensor type is added with an extra field. The existing pipeline doesn't know what to do with that field. Schema-on-read (typical for data lakes) handles this gracefully — old code ignores new fields. Schema-on-write (typical for relational databases) requires a migration to add the column.

Monitoring the pipeline

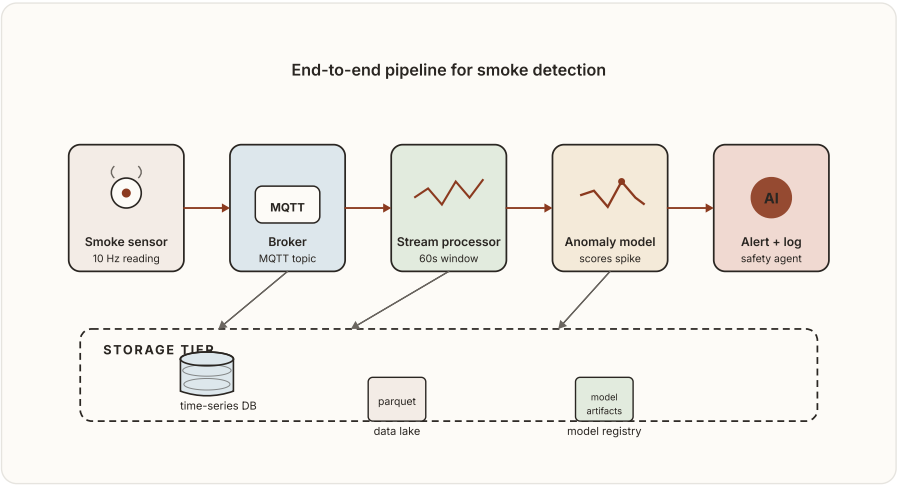
A data pipeline that silently fails is worse than one that fails loudly. Production-quality pipelines instrument themselves with metrics, and a monitoring system alerts when something looks wrong.

Four metrics worth tracking:

- **Data freshness.** Age of the most recent reading for each sensor. Alert if it exceeds three times the expected interval.
- **Value range.** A temperature below -50°C or above 100°C inside a Swedish office building is impossible. Alert when readings violate physical bounds.
- **Volume.** Number of readings per minute. A sudden drop signals offline sensors or a broken consumer.
- **Error rate.** Number of readings rejected by validation. A spike suggests a schema change or a faulty sensor.

The standard stack for this is **Prometheus** (a metrics database that scrapes endpoints periodically) plus **Grafana** (a dashboard tool that visualises the metrics and triggers alerts). Both are free, both run in Docker, both are industry standard.

Part 12 — A Worked Example: Smoke Detection Data Pipeline



Every component of the smoke-detection data pipeline: sensors at the top push readings down through ingestion, processing, modelling, and alerting — and everything is persisted to storage for replay and training.

To make all of the above concrete, follow one use case — smoke detection — through every stage.

Step 1: Ingestion. Each smoke sensor (one per room) publishes a reading every 5 seconds. The reading is a JSON object:

```
{
  "ts": "2026-04-27T13:42:00.250Z",
  "sensor_id": "smoke-A2306",
  "room": "A2306",
  "level": "level1",
  "type": "smoke",
  "unit": "fraction",
  "value": 0.04
}
```

Published to MQTT topic `sensors/level1/A2306/smoke` at QoS 1.

Step 2: Stream consumer. A small Python process subscribes to `sensors/#`. For every incoming message, it inserts a row into the TimescaleDB `readings` hypertable (the hot path) and appends to today's Parquet file in MinIO under `s3://building-data/bronze/smoke/date=2026-04-27/readings.parquet` (the cold path).

Step 3: Stream processing. A second small process keeps an in-memory 5-minute sliding window per sensor. For each new reading, it recomputes the count of readings above 0.5 in that window. If the count exceeds a threshold, it publishes to `alerts/anomaly/A2306`.

Step 4: Anomaly model (real-time). The safety agent subscribes to `alerts/anomaly/+`. When an alert arrives, it queries TimescaleDB for the last 60 seconds of smoke, temperature, and CO2 in that room, computes a feature vector, and runs the Isolation Forest model. If the score is above 0.8, it issues a sprinkler command.

Step 5: Batch training (nightly). A scheduled job runs at 3 a.m. each night. It reads the previous 24 hours of bronze data with DuckDB, cleans it (silver), computes ML features (gold), and writes the result to `s3://building-data/gold/smoke-features/date=2026-04-27/`. The training script reads the gold file, trains an updated anomaly model, and writes the new model to a model store. The safety agent picks up the new model on its next restart.

Step 6: Monitoring. A dashboard in Grafana shows per-sensor freshness, the readings-per-minute rate, the false-positive rate of the anomaly model over the last week, and the percentage of readings rejected by validation. An alert fires when any sensor's freshness exceeds 30 seconds or when the false-positive rate exceeds 5 percent.

That sequence is a complete, production-shaped data pipeline. Bronze, silver, gold, hot path, cold path, monitoring. Every step uses tools that fit on a laptop. Every step generates artefacts that survive across restarts.

Part 13 — Vocabulary Reference

Every term used in this chapter, defined.

TERM	DEFINITION
Data engineering	The discipline of designing, building, and maintaining the systems that move data from sources to consumers
Ingestion	The act of getting data from a producer (sensor) into a durable store
Stream processing	Operating on data as it arrives, one event at a time, without first storing it
Batch processing	Operating on large piles of stored data all at once on a schedule
Window (in streaming)	A time-bounded subset of a stream over which aggregations are computed
Tumbling window	A non-overlapping window of fixed size
Sliding window	An overlapping window of fixed size that updates continuously
Session window	A dynamic window that closes when events stop arriving for a gap
Complex event processing (CEP)	Detecting temporal patterns across multiple events
Data lake	A storage system that keeps raw data in its native format until it is needed
Medallion architecture	A pattern that organises a data lake into bronze (raw), silver (cleaned), gold (features), and model zones
Bronze layer	The raw, immutable copy of incoming data
Silver layer	Cleaned, deduplicated, validated data with a unified schema
Gold layer	ML-ready features derived from silver
Parquet	A columnar binary file format, compressed and schema-enforced, used for data lakes

TERM	DEFINITION
CSV	Comma-separated text file format; simple but slow
JSON Lines (JSONL)	One JSON object per line; flexible schema for event logs
Schema on write	The schema is enforced when data is stored (e.g., relational databases)
Schema on read	The schema is applied when data is queried (e.g., data lakes)
Object storage	A storage system that holds files (objects) in flat buckets, with HTTP API
MinIO	An open-source object storage system compatible with the Amazon S3 API
Time-series database (TSDB)	A database optimised for time-stamped data, append-heavy writes, and time-range queries
InfluxDB	A purpose-built time-series database popular in IoT
TimescaleDB	A PostgreSQL extension that adds time-series capabilities
Hypertable	A TimescaleDB construct that partitions data by time automatically
DuckDB	An in-process SQL engine that reads Parquet files directly
ClickHouse	A column-oriented analytical database for read-heavy workloads
ETL (Extract, Transform, Load)	A pipeline that reads data, reshapes it, and writes it to a downstream store
Feature engineering	The process of converting raw data into informative signals for ML
Rolling average	The average of a value over a recent window of time

TERM	DEFINITION
Cyclical encoding	Representing periodic variables (hour, day-of-week) using sine and cosine to preserve their adjacency
Feature store	A database that holds ML-ready features ready to be consumed by training and inference
dbt	A tool for expressing data transformations as SQL queries with built-in testing
Apache Airflow	A workflow orchestration platform that runs ETL pipelines as scheduled DAGs
NTP (Network Time Protocol)	A protocol for synchronising clocks across networked devices
Prometheus	A metrics database that scrapes data from instrumented services
Grafana	A dashboard tool that visualises metrics and triggers alerts
Idempotent insert	An insert that produces the same result whether performed once or many times

Part 14 — Summary in Five Sentences

1. Sensor data has a shape unlike most software data: high volume, fast velocity, mixed variety, and a hard tension between real-time decisions, ML training, and long-term analytics.
2. Ingestion uses a message broker as a buffer between sensors and storage, so that producers and consumers are independent and a slow downstream system does not lose data.
3. Storage is split between a hot path (TimescaleDB for real-time queries) and a cold path (Parquet on MinIO for historical analytics), with the medallion architecture (bronze, silver, gold, model) organising the cold path.

4. Feature engineering — rolling averages, rate of change, cyclical time encodings, cross-sensor relationships — turns raw readings into signals the ML model can learn from.
5. Data quality is a safety concern, not just an analytics concern; pipelines must detect missing, duplicate, stale, and clock-drifted data, and they must instrument themselves with metrics that signal when something is going wrong.

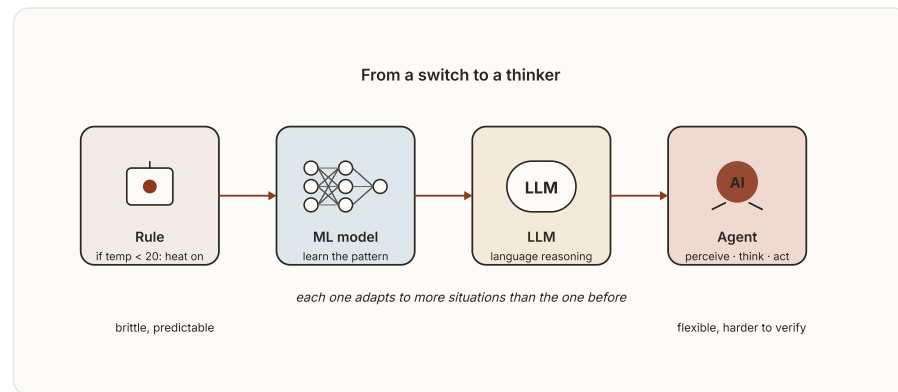
These five ideas are the foundation for every downstream use of data in this course — for AI agents, dashboards, training runs, and audit trails.

Lecture 4 — Agentic AI for Autonomous Systems

Standalone notes for the fourth lecture of D7065E. Read on its own or alongside

`lectures/lecture-4-agentic-ai.md`.

Part 1 — From a Switch to a Thinker: Three Generations of Automation



Four generations of automation, each adapting to more situations than the one before. The trade-off is always: more flexible behaviour, harder to predict.

The history of building automation is the history of decisions getting smarter. Three generations of automation describe where the field has been and where it is now.

Generation 1: Rule-based automation

The oldest and simplest. A rule says: `if temperature > 25°C then turn on the AC`. A programmable logic controller (a small industrial computer) can evaluate thousands of these rules per millisecond. The behaviour is deterministic, predictable, and easy to verify. Engineers love rules because rules don't surprise them.

A useful image: a thermostat with a dial. You set the dial, the thermostat compares the room temperature against the dial, and the heater clicks on or off. There is no thinking happening. There is no context. The thermostat doesn't know it's three in the morning, or that the building is empty, or that today is a public holiday. It only knows the number.

Rules work beautifully until reality stops fitting into the rules. A rule that turns on the AC at 25°C is correct on most days. It is wrong at 3 a.m. when nobody is in the building. It is wrong on a holiday when the office is closed. It is wrong when a faulty ventilation duct is causing the heat — running the AC will only make the duct fault worse. Real buildings have hundreds of interacting rules, each trying to cover an edge case, and the interactions between them become impossible to maintain.

Generation 2: Machine-learning inference

A learned model replaces hand-crafted rules. Instead of `if temperature > 25°C`, the model takes a list of features — temperature, occupancy, time of day, day of week, weather forecast, recent HVAC commands — and outputs a prediction. The model is trained on thousands or millions of examples of how the building behaves, and it learns patterns no rule-writer would have thought to capture.

A useful image: a doctor who has seen ten thousand patients. The doctor doesn't follow a single rule like "if temperature > 38°C, prescribe antibiotics." They take in a constellation of symptoms — temperature, blood pressure, the patient's history, the time of year — and recognise the pattern. The diagnosis comes from pattern recognition, not from a flowchart.

ML inference is much better than rules at handling situations the engineer didn't anticipate. But it is still, fundamentally, a function: input features in, prediction out. The model does not reason about *why*. It cannot explain itself in plain language. It cannot handle situations far outside its training data.

Generation 3: Agentic AI

Agentic AI goes one step further. An **agent** observes the current situation, reasons about its context in natural language, plans a sequence of actions, calls tools to execute the plan, observes the outcome, and adjusts its approach. The brain of the agent is typically a large language model (an LLM).

A useful image: instead of a doctor following a flowchart, imagine a senior consultant. The consultant asks the patient questions, weighs trade-offs out loud, considers what would happen if they chose treatment A versus treatment B, runs a test to gather more information, and explains

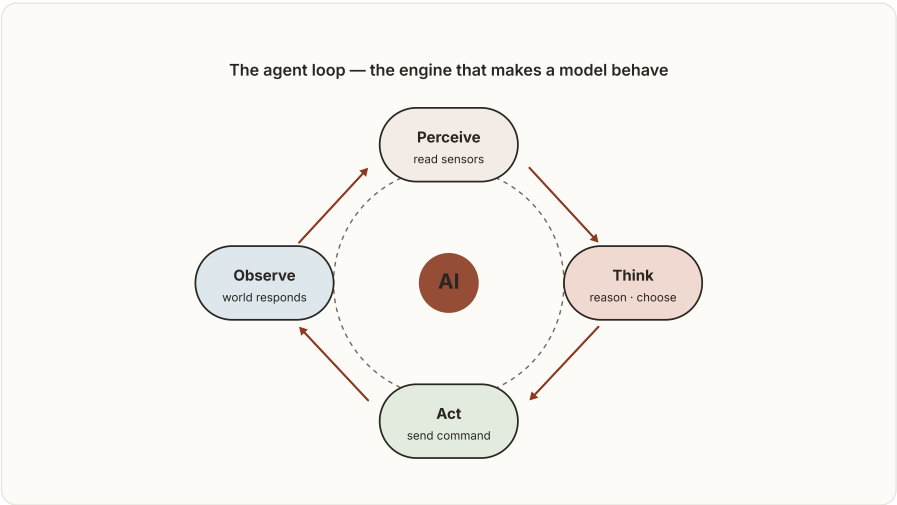
the reasoning to the patient in plain English. The consultant might encounter a case they have never seen before and still produce a sensible plan by reasoning from first principles.

An LLM-based agent can:

- Handle a situation it has never encountered in training.
- Explain its decisions in human language.
- Ask for clarification when the input is ambiguous.
- Adjust its plan after observing the result of an action.

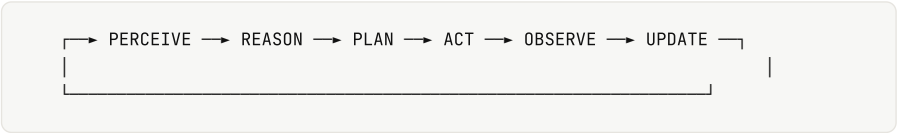
These three generations are not in competition. A real building uses all three: hard-wired rules for the most safety-critical functions (because rules are fast and verifiable), ML models for real-time inference (because models are smarter than rules), and LLM agents for higher-level reasoning that needs context and explanation (because agents can think). The art is knowing which to use where.

Part 2 — The Agent Loop



The agent loop is the engine that turns a model into a behaviour. Perceive, think, act, observe — and round again.

Every agent, no matter the framework or the model, runs the same loop. Six steps, repeated forever.



A useful image: a chess player thinking through a move. They look at the board (perceive). They reason about what's threatening and what's possible. They plan a sequence — capture the knight, then move the rook. They make the first move (act). They watch what the opponent does (observe). They update their mental notes about the opponent's style (update). Then the loop starts again with the new board state.

Perceive

The agent gathers fresh information about its environment. For a building agent, this means querying the time-series database for recent sensor readings, checking the dashboard's pending alerts, looking at what other agents have done recently.

A useful image: walking into a room and noticing the temperature, the people present, the open door, the smell of food. You're not deciding anything yet — you're just taking it in.

Reason

The agent thinks through what the perceptions mean. With an LLM-based agent, this is literally the LLM generating a "Thought" — a short natural-language sentence explaining its interpretation.

A useful image: muttering to yourself as you walk through the room. "Hot in here. Three people. Door's open to the hot corridor — that's probably why. They're working, not just passing through — I don't want to interrupt them."

Plan

The agent decides on a sequence of actions that should achieve its goal. The plan might be one action ("close the door") or several ("close the door, increase ventilation, send a notification").

A useful image: drawing up a small to-do list. "Step one: close the door. Step two: check if it helped in five minutes."

Act

The agent executes the plan by calling tools — functions that affect the world. A tool might command an actuator, query a database, send an alert, or compute a route.

A useful image: actually walking over and closing the door. Up until now everything has been in your head. Now you put your hand on the handle.

Observe

After acting, the agent checks what happened. Did the actuator state change? Did the sensor reading respond? Was the alert sent?

A useful image: stepping back and watching the temperature display for a minute. Did closing the door cool the room?

Update

The agent writes the result of this loop into its memory or audit log, so the next loop has the benefit of this experience. Without the update step, every loop starts from a blank slate.

A useful image: jotting in your notebook, "closed the door in room A2306 at 14:32, room temperature dropped 1.2°C in five minutes, conclude that the corridor was indeed the heat source." Tomorrow, you'll skip the diagnostic and close the door faster.

The update step is the one that turns inference into learning. Many beginner agents skip it, and they become elaborate functions that don't actually get smarter over time. A well-designed agent always closes the loop with a structured note that the next perceive step can read.

Part 3 — Choosing the Right Tool: Rules vs ML vs Agents

A useful question to ask before any decision is "how fast must this be, and how much context does it need?" The answer points to one of three approaches.

DECISION TYPE	LATENCY BUDGET	EXAMPLE	BEST APPROACH
Safety-critical	Under 100 ms	Fire suppression, emergency unlock	Hard-coded rule, possibly with ML on the edge
Real-time control	Under 1 second	Adjust temperature setpoint	Edge ML inference
Operational reasoning	1 to 30 seconds	Optimise HVAC schedule for the next hour	LLM agent
Strategic planning	Minutes	Plan tomorrow's energy schedule	LLM agent + optimiser
Human-facing explanation	Interactive	Explain why heating changed	LLM

A useful image: a hospital emergency department. The triage nurse follows a strict rule ("airway, breathing, circulation") because seconds count. The resident on call uses pattern recognition from training. The senior consultant reasons through complex cases. Three layers, three levels of cost and speed, each used where it fits.

A well-designed building control system uses all three layers in the same way:

- **Rules** for the fastest safety functions, where the cost of being slow is human harm and the cost of being wrong is catastrophic.
- **ML models** for real-time inference, where the situation is well-defined enough to train on and the latency budget rules out anything slower.
- **LLM agents** for higher-level reasoning, where context, trade-offs, and explanation matter more than raw speed.

Part 4 — Why LLMs Make Good Controllers

Large language models bring four capabilities to building control that neither rules nor traditional ML can match. Each is worth understanding because it shapes when an LLM is the right tool.

Contextual reasoning

An LLM can reason about trade-offs that involve many objectives at once. Consider this: "Energy cost is currently high because of peak pricing, but three occupants are complaining about the heat, and the outdoor temperature is forecast to drop in two hours. The optimal action is to tolerate the discomfort for 45 minutes, then pre-cool aggressively when the price drops." No rule system can express this kind of multi-dimensional weighing. No ML classifier can either, because the relevant facts are different every time.

A useful image: a chess grandmaster commenting on a position. "I'd normally trade queens here, but my opponent has shown they prefer endgames, and I'm slightly down on time — so I'll keep the queens and look for a tactical opportunity instead." The reasoning weaves together many considerations that no fixed rule could capture.

Natural-language understanding

A building manager doesn't think in terms of CO2 sensor IDs. They think in terms of "the meeting room on the third floor feels stuffy." An LLM agent can take that sentence, translate it into a sensor query, diagnose the issue, and act — without anyone having to look up the sensor's ID.

A useful image: a smart assistant who understands "the room that gets afternoon sun" without being told which compass direction that is.

Explainability

When an LLM makes a decision, it can explain it in a paragraph. "I turned off HVAC zone 4 because the CO2 sensor in that zone has been stuck at 450 ppm for the last 30 minutes despite three occupants being present according to the booking system, which suggests a sensor fault rather than genuine air quality. I have created a maintenance ticket." A traditional ML model cannot produce this. It outputs a number, not an explanation.

A useful image: the difference between a thermometer that beeps and a thermometer that says, "I'm beeping because the patient's fever rose 0.8°C in the last hour, faster than the usual recovery curve, so we should check for infection."

Handling novel situations

A fire alarm in a building hosting a film shoot that uses artificial smoke. A CO2 sensor that has been painted over by accident. A building manager who manually overrode the HVAC last week and forgot. LLMs handle these because they reason from common sense. Rule systems and ML models fail because none of these are in the rules or the training data.

A useful image: a new employee on their first day, facing a situation nobody trained them for. They reason from first principles: "this seems unusual; let me check with someone before acting." That kind of cautious common sense is exactly what an LLM brings.

Part 5 — The Limits of LLMs (and How to Work Around Them)

LLMs are powerful but flawed. Four problems matter for building control, and each has a known mitigation.

Hallucination

LLMs sometimes produce plausible-sounding but incorrect outputs. An LLM might suggest activating an actuator that doesn't exist, claim a sensor is showing a value it isn't, or invent a regulation. Hallucination is not malice — the LLM is doing exactly what it was trained to do, which is produce text that *looks* right.

A useful image: a confident but unreliable witness in court. They speak with conviction. They're sometimes completely correct. They're sometimes completely wrong, and they don't know the difference.

Three mitigations:

- **Constrain the output to a JSON schema.** The LLM is forced to produce structured output with valid field types. It can't invent random tools or actuators because those aren't in the schema.
- **Validate every tool-call argument before execution.** The LLM might say "activate sprinkler in room ZZZ." The validation layer rejects this because room ZZZ doesn't exist. The LLM never reaches the actuator.
- **Never trust the LLM's claims about sensor values.** Always read ground truth from the API or database. If the LLM says "the temperature is 32°C," check the database. The LLM is the reasoner, not the source of truth.

Latency

A cloud LLM API call takes 500 to 3000 milliseconds. That is far too slow for any decision on the critical safety path.

A useful image: imagine a thermostat that has to consult its lawyer before clicking on. The lawyer's advice might be excellent, but it arrives after the room has overheated.

Mitigations:

- **Use LLMs only for deliberative reasoning.** Decisions that can tolerate seconds of latency — HVAC scheduling, anomaly classification, planning — are fine.
- **Use ML models for real-time inference.** The ML model runs in milliseconds. When the ML model flags an anomaly, *then* the LLM is invoked to reason about whether it's a true positive.
- **Cache common reasoning patterns.** "Smoke above 0.7 plus rising temperature" is a frequent input. Cache the LLM's response.
- **Use smaller local models for routine decisions.** A local model like Llama 3.2 3B running on a GPU server takes 100-200 ms instead of 1000-2000 ms.

Cost

Calling a cloud LLM API for every sensor reading is economically unsustainable. A building with 100 sensors reading every five seconds would make 1.7 million LLM calls per day. Even at one cent per call, that's \$17,000 a day.

A useful image: you would not call a senior lawyer to review every email you send. You'd ask the lawyer about the hard ones and handle the rest yourself.

Mitigations:

- **Invoke the LLM only for novel or anomalous situations.** A simple ML or rule filter handles routine cases. The LLM is reserved for the unusual.
- **Use the ML model as a filter that escalates to the LLM only when needed.** This is the most common production pattern. Inference runs on every reading; the LLM runs only on the few that look interesting.

Local models

A local model — running on the lab's GPU server or on a developer's laptop — solves both the latency and the cost problem. The model is slower and less capable than a frontier cloud model, but it's free and fast.

Ollama is the standard tool for running open-source LLMs (Llama, Gemma, Mistral, Qwen) locally with a simple API. The course's lab GPU server runs Ollama. For routine reasoning in a building control system, a local model is entirely adequate.

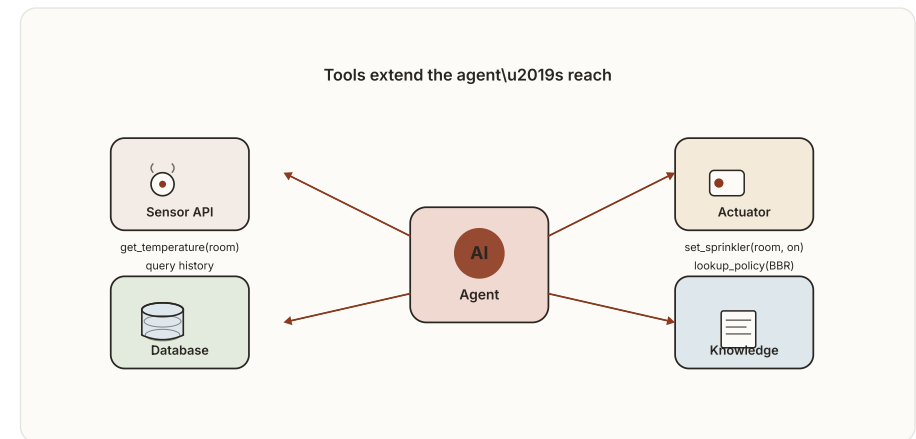
A useful image: having a librarian in the office building. They don't know everything that the New York Public Library would know, but they're available immediately and free.

Structured output

The single most useful trick for working with LLMs reliably is to ask for JSON instead of free text. With OpenAI-compatible APIs (including Ollama), `response_format={"type": "json_object"}` forces the model to produce JSON. With Anthropic's API, `tool_choice="required"` forces a tool call that has a defined schema.

A useful image: asking a job applicant to fill in a form instead of writing a letter. The form has named fields and types. Even a poor candidate produces something parseable. A letter, by contrast, can be free-form and impossible to process automatically.

Part 6 — Giving the Agent Hands: Tool Use and Function Calling



Tools let the agent reach into systems it can't operate by itself. Each tool is a typed function the model can call when it needs to act.

An LLM, on its own, can only read text and produce text. It can't read a sensor, write to a database, or command an actuator. **Tool calling** — also called function calling — is the mechanism that extends the LLM with the ability to invoke external functions.

A useful image: a brilliant consultant who is locked in a room with only a phone. They can think and talk. They can call you and ask you to do things. You go and do those things, then come back and report what happened.

The protocol works in five steps:

1. The system defines a set of tools (functions) with names, descriptions, and parameter schemas.
2. The user sends a message to the LLM, along with the tool definitions.
3. The LLM reasons about what to do. If it needs to act, it generates a structured **tool call** — a JSON object with the function name and the arguments.
4. The calling code executes that function and gets a result.

5. The result is returned to the LLM as a **tool result** message. The LLM then either calls another tool or produces a final answer.

This loop — Thought, Action, Observation, Thought, Action, Observation — is the foundation of the **ReAct pattern**, introduced by Yao et al. in 2022. ReAct stands for "Reasoning + Acting", and it is by far the most common pattern for LLM agents.

A worked example:

```
Thought: The smoke sensor in A2306 reads 0.82, which is above the alert
threshold of 0.7. I should check if there are corroborating
signals before activating sprinklers.
Action: read_sensors(room="A2306", sensor_types=["temperature", "co2"])
Observ: temperature=38.5°C (normal is 21°C), co2=890 ppm (normal is 450)
Thought: Temperature is significantly elevated and CO2 is rising. Combined
with smoke, this is consistent with a fire, not a false alarm.
I should activate sprinklers and unlock evacuation doors.
Action: set_actuator(actuator_id="sprinkler-A2306", state="on",
                    reason="fire detected: smoke=0.82, temp=38.5C")
Observ: {success: true, state: "on"}
Action: set_actuator(actuator_id="doors-floor2", state="unlocked",
                    reason="fire evacuation")
Observ: {success: true, state: "unlocked"}
Thought: Sprinklers activated and evacuation doors unlocked. I should
alert the building manager.
Action: create_alert(severity="critical", message="Fire detected in
A2306. Sprinklers activated. Doors unlocked.")
```

The key insight is that the reasoning (Thought) is explicit. Every step has a sentence explaining why the next action is justified. For safety-critical systems this is the difference between a system you can audit and one you can't. Every decision the agent makes can be replayed and explained.

A useful image: a detective's case notebook. Every observation, every deduction, every action is written down in order. If the case goes to court, the notebook is the evidence.

Part 7 — Designing Good Tools

Tool design is more important than agent design. A well-designed tool catalogue lets even a small LLM reason effectively. A badly-designed catalogue confuses even a frontier model. Four principles drive good tool design.

One tool per concern

Don't write one big `control_building()` tool that does everything. Write narrow tools, each with a single job: `read_sensors`, `set_actuator`, `find_route`, `highlight_rooms`, `query_history`, `create_alert`. Each is small, validated, and easy to test.

A useful image: a Swiss Army knife with named tools versus a single blob marked "do something useful." When the LLM needs to cut wire, it picks the wire cutter. When it needs to open a bottle, it picks the bottle opener. Each tool's purpose is obvious from its name.

Descriptive names and docstrings

The LLM decides which tool to call by reading its name and description. A tool called `get_data()` will be misused. A tool called `read_room_sensors(room_id, sensor_type, last_minutes)` will be used correctly. Spend time on the description — it is literally what teaches the LLM how to use the tool.

A useful image: jars in a kitchen pantry. If they're all labelled "stuff," the cook will pick at random. If each one is labelled clearly — "sea salt, fine" or "bicarbonate of soda" — the cook will reach for the right one.

Validate inputs before execution

Never trust tool-call arguments unconditionally. Check that the room exists, that the sensor type is valid, that numeric arguments are within sensible ranges. If validation fails, return an error message — the LLM can read it and try again with corrected arguments.

A useful image: a bouncer at a club door. The bouncer doesn't care that the person is wearing a tuxedo. They check the guest list. If the name isn't on the list, the person doesn't get in, regardless of how confident they appear.

Return rich, informative results

When a tool returns a result, the LLM uses that result to reason about what to do next. A bare result like `{"state": "on"}` is the bare minimum. A rich result is far more useful:

```
{
  "state": "on",
  "previous_state": "off",
  "last_changed_at": "2026-04-27T13:42:05Z",
  "commanded_by": "safety_agent",
  "applied_within_ms": 230
}
```

The LLM can now reason about how recently the actuator changed, who else has commanded it, and whether the response time was normal.

A useful image: a good waiter doesn't just bring you the dish. They tell you what's in it and what it goes well with. The extra context lets you order better the next time.

A complete example

A tool definition that follows all four principles, in OpenAI/Ollama function-calling format:

```
tools = [{
  "type": "function",
  "function": {
    "name": "read_sensors",
    "description": (
      "Read current sensor values for a specific room. "
      "Returns temperature (°C), humidity (%), CO2 (ppm), "
      "smoke (0-1), and occupancy state."),
    "parameters": {
      "type": "object",
      "properties": {
        "room_id": {
          "type": "string",
          "description": "Room identifier, e.g. 'A2306'"
        },
        "sensor_types": {
          "type": "array",
          "items": {"type": "string"},
          "description": ("List of sensor types to read. "
            "Options: temperature, humidity, "
            "co2, smoke, occupancy")
        }
      },
      "required": ["room_id"]
    }
  }
}]
```

Part 8 — The Model Context Protocol (MCP)

A new agent project usually starts with the team defining their own tool format, their own way of registering tools, their own way of exposing them to the LLM. After a few projects, everyone has invented the same wheel a slightly different way.

The **Model Context Protocol**, abbreviated MCP, was proposed by Anthropic in 2024 to standardise this. It defines a single protocol for connecting AI models to external data sources and tools, so that any MCP-compatible model can discover and use tools from any MCP-compatible server.

The architecture has three components:

- **MCP Host** — the application that runs the LLM (Claude Code, an AI agent application).
- **MCP Client** — the protocol client inside the host that manages server connections.
- **MCP Server** — a server that exposes tools, resources, and prompts via the MCP protocol.

A useful image: think of MCP as a universal remote control standard. Before MCP, every TV maker shipped its own remote, and you needed a dozen remotes for your living room. MCP is HDMI-CEC for AI tools — one standard so that any device works with any controller.

For building control, an **MCP server** wraps the BuildSim API as MCP tools. The BuildSim MCP server in the course repository exposes 17 tools: `get_building`, `get_rooms`, `list_equipment`, `add_sensor`, `set_actuator_state`, `highlight_rooms`, `find_route`, and more. Any MCP-compatible agent — Claude, GPT-4, a local Ollama model with an MCP wrapper — can discover and use those tools automatically. No custom integration code.

The protocol itself uses JSON-RPC 2.0 over stdio or HTTP with server-sent events. Implementing an MCP server means defining tool schemas, implementing tool handlers (which usually wrap REST API calls), exposing a `tools/list` endpoint that returns the catalogue, and exposing a `tools/call` endpoint that executes a chosen tool.

The MCP Python SDK handles the protocol plumbing. Only the tool implementations have to be written.

Part 9 — Agent Frameworks: What They Buy You and What They Cost

Agent frameworks provide scaffolding for building LLM agents: tool execution loops, memory management, state machines, multi-agent coordination, and integrations with many LLM providers. The right choice depends on the complexity of the use case.

No framework

For simple agents that call one or two tools in a fixed sequence, plain Python with direct LLM API calls is often clearest. A 50-line Python script that reads sensors, constructs a prompt, calls the LLM, parses the tool call, executes the tool, and loops is simpler to understand, test, and debug than the same logic expressed in a framework's domain-specific language.

A useful image: cooking a single fried egg. You don't need a chef. You need a frying pan and basic technique.

LangChain

The most widely used LLM application framework. Provides chains (sequential tool calls), agents (LLM-driven tool selection), memory (conversation history), and a huge ecosystem of integrations. Best when many integrations are needed quickly — connectors to databases, vector stores, cloud LLM providers, and so on.

A useful image: a well-stocked kitchen. Many tools, many ingredients, many recipes already half-prepared. Great for many dishes; overkill if you only ever fry eggs.

LangGraph

A graph-based agent framework from the LangChain team. Agents are defined as nodes in a directed graph with cycles; the LLM decides which edge to follow at each step. Better than plain LangChain for complex workflows with branching, looping, and human-in-the-loop checkpoints.

```

from langgraph.graph import StateGraph, END
from langgraph.prebuilt import ToolNode

class AgentState(TypedDict):
    messages: list
    sensor_readings: dict
    pending_actions: list

workflow = StateGraph(AgentState)
workflow.add_node("read_sensors", sensor_reading_node)
workflow.add_node("reason", llm_reasoning_node)
workflow.add_node("act", ToolNode(building_tools))

workflow.set_entry_point("read_sensors")
workflow.add_edge("read_sensors", "reason")
workflow.add_conditional_edges("reason", should_act,
                               {"act": "act", "done": END})
workflow.add_edge("act", "reason")

agent = workflow.compile()

```

A useful image: a board game with a movement map. Each square has rules for what moves are legal. The agent rolls the dice and moves; the framework enforces the rules of the board.

CrewAI

A framework for role-based multi-agent systems. Agents are defined with **roles** (architect, designer, builder), **goals**, and **backstories**. **Tasks** have descriptions and expected outputs. **Crews** assign tasks to agents, and agents can delegate to each other.

A useful image: a film production team. The director, the cinematographer, the editor — each has a defined role, each can collaborate with the others, and the producer (the crew) coordinates them.

AutoGen

Microsoft's conversational multi-agent framework. Agents exchange messages in a structured conversation; a coordinator directs the flow. Good for tasks where multiple agents critique each other — code review, report writing, multi-step debugging.

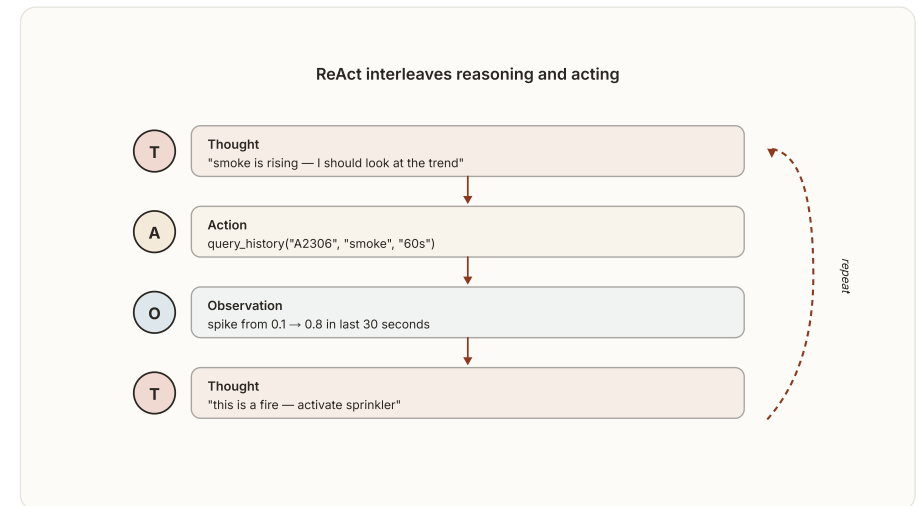
A useful image: a panel discussion. The moderator picks who speaks. Each panellist has a perspective. The conversation produces a richer answer than any single panellist alone.

Pydantic AI

A newer framework built around Pydantic for structured outputs and type safety. Good for agents that need reliable, typed tool-call arguments and responses. Less feature-rich than LangGraph but simpler to use for straightforward agents.

A useful image: building furniture with a precision template. Less flexible than freehand work, but every joint fits.

Part 10 — The ReAct Pattern in Detail



In the ReAct pattern, the agent alternates between reasoning (Thought) and acting (Action), with each Observation feeding the next Thought. The trace is also the explanation.

The ReAct pattern — Reasoning + Acting — was introduced by Yao et al. in their 2022 paper. It is short, clear, and has become the de facto template for LLM agents.

The pattern interleaves three step types:

- **Thought:** the LLM's reasoning, in natural language.
- **Action:** a tool call.
- **Observation:** the result of the tool call.

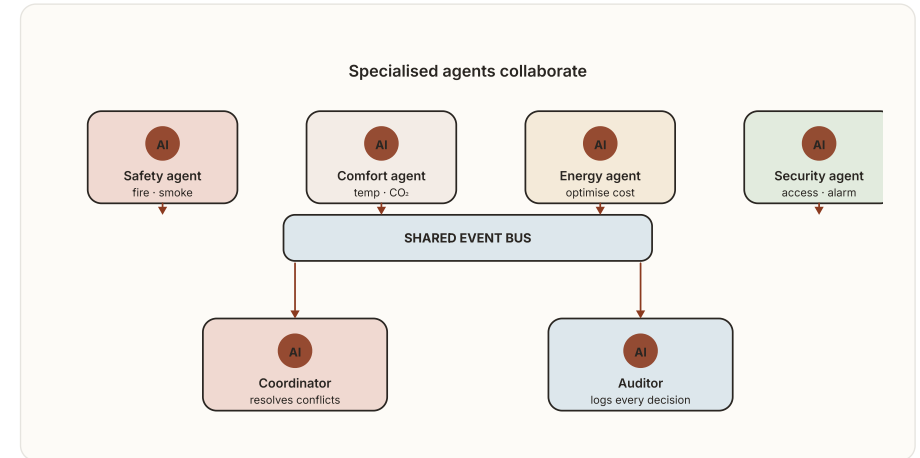
Repeated until the LLM produces a final answer instead of another action.

The key insight is that the **reasoning is explicit and auditable**. For a safety-critical building control system, this is invaluable. After a fire incident, you can replay every Thought, every Action, every Observation, and see exactly what the agent did and why.

A useful image: a pilot's flight log. Every decision is timestamped, with the reason. If the plane lands safely, nobody looks at the log. If something goes wrong, the log is the first thing the investigators read.

The interleaving also helps the LLM stay on track. A pure plan-then-execute pattern is fragile: the plan is made once, and if reality doesn't match the plan, the agent fails. ReAct adapts continuously — every Observation feeds back into the next Thought, and the plan evolves.

Part 11 — Multi-Agent Systems



Specialised agents own a slice of the problem and communicate through a shared bus. A coordinator resolves conflicts; an auditor records every decision.

A single agent that controls every aspect of a building is fragile. It is hard to test, hard to update, and has a single point of failure. Multi-agent systems decompose the control problem into specialised agents that coordinate.

Why multiple agents

Four reasons.

Separation of concerns. A **safety agent** handles fires and emergencies with highest priority. An **energy agent** optimises consumption. A **comfort agent** maintains temperature and air quality. Each agent has a clear responsibility and can be developed, tested, and updated independently.

Parallel reasoning. Multiple agents can reason simultaneously. The energy agent is continuously optimising the HVAC schedule while the comfort agent is processing an occupant feedback event, and they do not block each other.

Robustness. If the comfort agent crashes, the safety agent keeps running. Single-agent architectures have a single point of failure.

Specialisation. A small, fast model may be sufficient for routine energy optimisation, while a larger model is used only for complex safety decisions. Different agents use different models, optimising cost and latency.

A useful image: a hospital. The cardiologist handles hearts. The nephrologist handles kidneys. The emergency physician handles whatever walks in. Each specialist is excellent at their narrow domain. The patient benefits from all of them.

How agents talk to each other

Three communication patterns.

Shared state (the blackboard pattern). All agents read and write to a shared data store (the time-series database, a Redis instance, the BuildSim API). Agents observe each other's actions through state changes.

A useful image: detectives in a precinct office sharing a whiteboard. Each adds clues; each sees what the others have added. Simple and decoupled, but with the risk that two detectives independently chase the same lead.

Message passing. Agents send structured messages to each other through a message queue (MQTT, Redis Streams). Explicit, auditable, decoupled.

A useful image: a customer-service team where every escalation is a ticket. The customer-service rep tickets a manager; the manager tickets engineering; engineering tickets the customer-service rep with a solution. Every step is recorded.

Shared context in the LLM. For LLM-based agents, the entire conversation history (including other agents' decisions) is included in each agent's context. Each agent sees what the others have decided. Expensive in tokens but produces well-informed decisions.

A useful image: a war-room briefing where every participant hears every report. Slow but coherent.

Part 12 — Conflict Resolution and the Pathologies Multi-Agent Brings

The moment a system has multiple agents with different objectives, conflicts become possible. The energy agent wants the HVAC off. The comfort agent wants it on. Who wins?

Conflict-resolution strategies

Priority-based. Each agent has a fixed priority. The safety agent's priority is 1000 — it always wins. Energy is 10. Comfort is 20. When agents conflict, the higher-priority agent's command is executed. Simple, predictable, but inflexible.

A useful image: emergency services priority. When the fire alarm rings, the fire brigade takes over the road. Nothing else competes.

Auction-based. Agents bid for actuator control with a numeric priority. The agent with the highest bid wins. Bids can be dynamic — the safety agent bids 1000 only when smoke is detected, 0 otherwise. This gives flexibility based on context.

A useful image: an auction house. The dynamic bid lets a normally-quiet bidder suddenly out-shout everyone when something special comes up.

Consensus. Agents negotiate a compromise. The energy agent proposes a temperature setpoint; the comfort agent counter-proposes; they converge. More complex but produces better outcomes when neither agent needs to win completely.

A useful image: two negotiators meeting in the middle. Each starts from their preferred outcome and walks halfway. Both leave slightly unhappy and entirely satisfied.

Hierarchical (supervisor pattern). A supervisor agent receives all proposed actions from subordinate agents, reasons about conflicts, and produces a final unified action set. The supervisor has the most context but is a bottleneck.

A useful image: a project manager arbitrating between team members. Slower than letting them work it out, but produces consistent decisions.

Pathologies — failure modes that don't exist with a single agent

Multi-agent systems can fail in ways that single-agent systems cannot.

Oscillation. Agent A turns on the HVAC, which raises the temperature; Agent B turns it off to save energy; temperature drops; Agent A turns it on again. The loop repeats indefinitely.

A useful image: two thermostats with different setpoints, wired into the same room. Each fights the other forever. Neither room nor energy bill ever stabilise.

The prevention is **hysteresis** — refusing to change state unless the trigger condition has been true for at least N seconds. A shared decision log so each agent knows what the others recently decided also helps.

Deadlock. Agent A waits for Agent B to release an actuator; Agent B waits for Agent A to release a different one. Neither can proceed.

A useful image: two cars at a four-way stop, both waiting for the other to go.

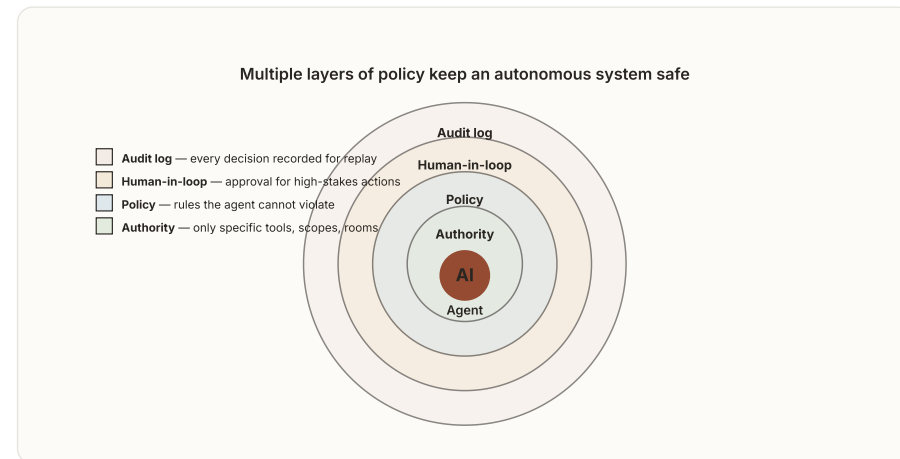
Prevention: avoid circular dependencies in actuator ownership; use timeout-based locks.

Thrashing. Multiple agents issue conflicting commands in rapid succession, causing actuators to change state faster than the physical system can respond.

A useful image: flipping a light switch ten times a second. The bulb never has time to warm up; it just flickers.

Prevention: rate-limit actuator commands; enforce a minimum time between state changes for any actuator.

Part 13 — Safety and Trust in Agentic AI



An autonomous agent is wrapped in concentric layers of policy. Authority limits what it can touch; policy says what it must never do; humans approve the rest; an audit log records everything.

An AI agent controlling a building can cause real harm. It can leave emergency doors locked during a fire, over-cool a server room, or cause energy spikes that trip a breaker. Safety in agentic AI is not abstract — it is an engineering requirement, and there are well-established techniques to enforce it.

Guardrails

Guardrails are constraints applied before an action is executed. The LLM never sees the guardrail; the validation code runs between the tool call and the actuator. Examples:

- Never lock all exits simultaneously, even if the LLM reasons that it should.
- Smoke suppression can only be deactivated by a human operator.
- Actuator commands above a certain rate are rejected.
- If any action would result in all HVAC units being off in winter, human confirmation is required.

A useful image: bumpers on a bowling lane. The LLM is the ball. The bumpers don't change the ball's spin or speed, but they catch a wild throw before it ends up in the gutter.

Human-in-the-loop (HITL)

For high-stakes, irreversible actions — activating fire suppression in a particular zone, unlocking emergency exits — the agent proposes the action and waits for a human to approve. HITL adds latency, but the cost of a wrong decision is high enough that the latency is worth it.

A useful image: a customer-service rep needing manager approval to issue a refund over £500. Slows things down, but stops large mistakes.

Audit trail

Every agent decision, tool call, and reasoning step must be logged immutably. When something goes wrong, reconstructing exactly what the agent decided and why is essential. The ReAct pattern is naturally audit-friendly: every Thought, every Action, every Observation is already a structured record.

A useful image: a bank ledger. Every transaction has a timestamp, an account, an amount, and a reference. If a discrepancy is found, the ledger is the source of truth.

Graceful degradation

When the AI agent is unavailable — the LLM API is down, the agent process has crashed, the network is degraded — the system must fall back to a safe state without human intervention. All actuators remain in their current state. Rule-based fallbacks handle the most critical safety functions. Alerts notify operators that AI control is inactive. The system can still be operated manually via the BuildSim API.

A useful image: a hospital's manual procedures when the IT system goes down. Paper charts. Phone calls. Slower, less informed, but the patients still get cared for.

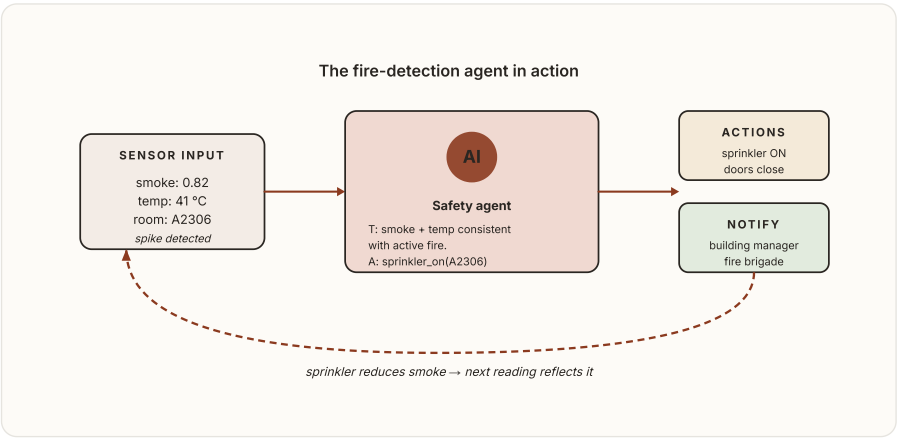
Testing safety

Adversarial testing is essential. Inject sensor faults: what does the agent do when the smoke sensor reads 0 for ten minutes? Simulate network outages: what happens when the LLM API returns an error? Send conflicting commands: what does the coordinator do? Test with novel situations: what if a sensor reports a value the agent has never seen before?

A useful image: a fire drill in an office. You don't wait for a real fire to find out whether the evacuation plan works.

Tests for these scenarios are written **before** deployment, not after. They are part of the validation step of MBSE.

Part 14 — A Worked Example: Fire-Detection Agent



The fire-detection agent in action: a sensor spike triggers reasoning, the agent calls actuator and notification tools, and the next sensor reading closes the loop.

To make all of this concrete, walk a fire-detection agent through one complete cycle.

Setup

- Smoke sensors per room, publishing readings every 5 seconds to MQTT.
- A consumer writing every reading to TimescaleDB.
- An anomaly model (Isolation Forest) that scores each sensor every 30 seconds.

- The safety agent, a ReAct agent using Ollama (Llama 3.2 3B locally) for reasoning.
- Six tools: `read_sensors` , `set_actuator` , `highlight_rooms` , `find_route` , `query_history` , `create_alert` .
- Guardrails: known actuator IDs only; legal states only; a `reason` field of at least 5 characters is required.
- Audit trail: every Thought, Action, Observation written to a `decisions` table in TimescaleDB.

The trigger

Smoke level in room A2306 spikes to 0.85. The anomaly model scores 0.91, well above threshold. The safety agent wakes for its perceive cycle.

Perceive

The agent calls `read_sensors(room="A2306", last_seconds=30)` . The result:

```
{
  "room": "A2306",
  "sensors": {
    "smoke":      {"current": 0.85, "mean": 0.62, "trend": +0.41},
    "temperature": {"current": 38.5, "mean": 32.1, "trend": +6.4},
    "co2":        {"current": 890,  "mean": 670,  "trend": +220}
  }
}
```

Reason

The agent generates a Thought:

"Smoke at 0.85, temperature rising sharply (+6.4 °C in 30 seconds), CO2 also rising. This is a high-confidence fire signal across three independent sensor types. I should activate sprinklers and unlock the evacuation doors. Before acting, I want to know the safest evacuation route."

Plan and act

The agent issues a tool call: `find_route(from_room="A2306", to_room="exit-stair-east")` . The result includes a path with five waypoints and a distance of 42 metres.

Then `set_actuator(actuator_id="sprinkler-A2306", state="on", reason="fire detected, smoke 0.85, temp 38.5, CO2 890, three signals concur")` . The guardrail checks that the actuator exists (yes), the state is legal (yes), and the reason is at least 5 characters (yes). The command goes through.

Then `set_actuator(actuator_id="firedoor-A2306-east", state="unlocked", reason="evacuation route to east stair")` .

Then `highlight_rooms(rooms=[{"room_id": 552, "color": "#ff0000"}, ...])` to make the fire zone visible on the 3D viewer.

Then `create_alert(severity="critical", message="Fire detected in A2306. Sprinklers active. Evacuation east via stair-east.")` .

Observe

The agent calls `read_sensors` again 30 seconds later. Smoke level has dropped to 0.45. Temperature still high but stable. The sprinkler is working.

Update

The full sequence — Thought, Actions, Observations — is written to the audit log. The agent's next cycle starts with this history available.

When something goes wrong

If Ollama is unreachable when the agent tries to reason, the agent falls back to a pre-coded rule: `if smoke > 0.7 then sprinkler on` . The decision is logged with `step='fallback'` so the audit trail records that the LLM was bypassed. A separate watchdog process enforces hard safety rules independent of the agent: smoke above 0.9 always triggers sprinklers, regardless of what any agent does.

Part 15 — Vocabulary Reference

Every term used in this chapter, defined.

TERM	DEFINITION
Agent	A system that perceives, reasons, plans, acts, observes, and updates — using an LLM, ML model, or rules as its reasoning engine
Agentic AI	An AI system built around an agent that uses an LLM to reason about its situation and choose actions
The Agent Loop	Perceive → Reason → Plan → Act → Observe → Update — the six-step cycle every agent runs
LLM (Large Language Model)	A neural network trained on text that can generate plausible text in response to prompts
Tool calling	The mechanism by which an LLM invokes external functions through structured tool-call messages
Function calling	Synonym for tool calling, used by OpenAI
ReAct pattern	Reasoning + Acting — interleaving Thought, Action, Observation in the agent's output
Thought	The LLM's natural-language reasoning step, logged for audit
Action	A tool call invoked by the LLM
Observation	The result of a tool call, returned to the LLM as input for the next step
Hallucination	The LLM producing plausible but incorrect output
Structured output	Forcing the LLM to produce JSON that matches a schema, rather than free-form text
Local model	An LLM running on local hardware (Ollama) instead of a cloud API
MCP (Model Context Protocol)	An open standard for connecting LLMs to external tools, proposed by Anthropic in 2024
MCP Host	The application that runs the LLM

TERM	DEFINITION
MCP Client	The protocol client managing connections to MCP servers
MCP Server	A server that exposes tools, resources, and prompts via MCP
Tool (in an agent)	A function the LLM can call to read or change the world
Tool schema	The JSON-schema definition of a tool's name, description, and parameters
LangChain	The most widely used LLM application framework, with chains, agents, memory, and many integrations
LangGraph	A graph-based agent framework from the LangChain team for complex workflows
CrewAI	A framework for role-based multi-agent systems
AutoGen	Microsoft's conversational multi-agent framework
Pydantic AI	A framework focused on structured outputs and type safety
Multi-agent system	An architecture with several specialised agents that coordinate to control one system
Coordination layer	The component that resolves conflicts between agents
Priority-based resolution	Each agent has a fixed priority; highest priority wins conflicts
Auction-based resolution	Agents bid for control; highest bid wins
Consensus resolution	Agents negotiate a compromise rather than one winning outright
Hierarchical (supervisor)	A supervisor agent arbitrates decisions made by subordinates
Oscillation	A pathology in which two agents flip an actuator back and forth

TERM	DEFINITION
Deadlock	A pathology in which two agents wait for each other forever
Thrashing	A pathology in which actuators change state faster than the physical system can respond
Hysteresis	Refusing to change state until the trigger condition has been true for a minimum time
Guardrail	A validation rule applied before an actuator command is executed
Human-in-the-loop (HITL)	Requiring human approval before high-stakes actions are executed
Audit trail	The immutable log of every agent decision, tool call, and reasoning step
Graceful degradation	The ability to fall back to a safe state when the AI agent is unavailable
Adversarial testing	Deliberately injecting faults to test how the agent handles them
Ollama	A tool for running open-source LLMs locally with a simple API

has a known mitigation through structured output, validation, ML pre-filtering, and local models.

- Tools are the agent's hands, and the four design principles — one tool per concern, descriptive names, input validation, rich results — matter more than the choice of framework or model.
- Multi-agent systems are stronger and more failure-resilient than single agents, but they require a conflict-resolution strategy (priority, auction, consensus, or hierarchy) and explicit mitigations for the pathologies they create (oscillation, deadlock, thrashing).

These five ideas are the foundation for every agent in the rest of the course.

Part 16 — Summary in Five Sentences

- Three generations of automation — rules, ML inference, and agentic AI — sit on a continuum from fast-and-rigid to slow-and-thoughtful, and a well-designed system uses all three where each fits.
- Every agent runs the same six-step loop — Perceive, Reason, Plan, Act, Observe, Update — and the Update step is what turns inference into learning.
- LLMs are powerful controllers because they handle context, novel situations, and natural language, but they hallucinate, are slow, and are expensive — every one of those weaknesses